# DICE: Decentralized Incentive-driven Federated Learning for Efficient & Secure Customization of Foundation Models in Web3

Hong Lin ⋆, Zhongle Xie ⋆, Lidan Shou ⋆ †, Ke Chen ⋆
⋆ State Key Laboratory of Blockchain and Data Security, Zhejiang University
{honglin,xiezl,should,chenk}@zju.edu.cn

## ABSTRACT

Customizing Foundation Models (FMs) for downstream tasks demands task-specific data, which burgeoning Web3 applications offer in vast and decentralized forms. Blockchain-powered federated learning (BCFL) has emerged as a promising privacy-preserving paradigm to securely customize models among *trustless* and *self-interested* Web3 participants. Yet, existing BCFL algorithms struggle with severe *efficiency* and *scalability* challenges arising from the tremendous computation and memory demands of FMs and the hardware-constrained nature of most participants. This paper addresses this critical gap by introducing DICE, the first BCFL algorithm devised for FM customization (FMC) tasks in Web3 applications. DICE enhances the typical BCFL paradigm with four integrated extensions: (1) a *dynamic progressive learning* scheme to enable hardware-constrained workers, especially those having abundant task-specific data, to produce updates of FMs; (2) a *model slice verification* scheme to enable hardware-constrained validators to efficiently detect and reject incorrect updates produced by malicious workers; (3) a *tree-structured committee consensus* algorithm to minimize the complexity of evaluating and aggregating massive updates while preserving security against malicious behavior; (4) an *efficiency-driven incentive scheme* to encourage participants to behave honestly and accelerate FMC tasks. We provide theoretical analyses of DICE's efficiency and security. Extensive experimental results on both vision and language FMC tasks confirm the effectiveness, efficiency, and security of DICE.

**PVLDB Reference Format:**
Hong Lin ⋆, Zhongle Xie ⋆, Lidan Shou ⋆ †, Ke Chen ⋆. DICE: Decentralized Incentive-driven Federated Learning for Efficient & Secure Customization of Foundation Models in Web3 . PVLDB, xxx(xxx): xxx - xxx, 2026.
doi:xxxx

**PVLDB Artifact Availability:**
The source code, data, and/or other artifacts have been made available at URL_TO_YOUR_ARTIFACTS.

## 1 INTRODUCTION

Pretrained Foundation Models (FMs) and Web3 are converging to create more intelligent, automated, and efficient decentralized systems. These pretrained models require task-specific **customization** through further pretraining or fine-tuning before serving downstream tasks [10, 78, 101]. Meanwhile, the rapid growth of Web3 applications generates vast amounts of decentralized data [7, 24, 55, 80, 116], which provides immense potential for foundation model customization (FMC). For instance, large-scale Web3 platforms such as Filecoin [23] and the Graph [27] produce terabytes or even petabytes of data spread across millions of peer-to-peer devices. However, exposing decentralized data may violate increasingly stringent data protection regulations [88]. Moreover, Web3 participants are *self-interested* and *trustless*: they are reluctant to contribute resources without economic incentives and may disrupt applications if malicious behavior yields profit [43, 49, 95, 104, 113].

Blockchain-powered Federated Learning (BCFL) [42, 47, 56, 56, 63, 64, 69, 115] has emerged as a promising paradigm to support privacy-preserving FMC among trustless Web3 participants. As shown in Fig. 1, a typical BCFL-based FMC pipeline involves three roles: a *model requester*, who needs a customized FM; *workers*, who locally train models using private data and submit *updates* (the model parameters changed during training); and *validators*, who verify and aggregate these updates. BCFL systems generally use blockchain networks such as Ethereum [13, 96] to coordinate participants through smart contracts for learning models by iterative

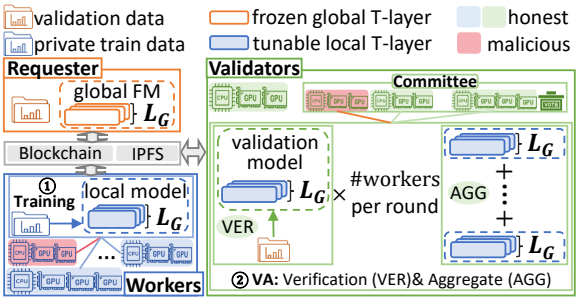



**Figure 1: Apply existing BCFL paradigm to customize BERT-Large [20] containing $L_G = 24$ Transformer layers (T-layers).**

| Batch Size | 8 | 16 | 24 | 32 | 64 | # Workers Per Round | 10 | 30 | 50 | 70 | 100 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| $MC_w$ (GB) | 24 | 39 | 55 | 71 | 135 | $MC_v$ (GB) | 24 | 72 | 120 | 168 | 240 |

**Table 1: A worker's and a validator's minimum memory cost per round (denoted by $MC_w$ and $MC_v$, respectively) for engaging FMC in Fig. 1. These statistics are approximated based on previous papers [31, 40, 62, 92] and the common industrial practice: using FP32 for FM training and FP16 for inference.**

† Lidan Shou is the corresponding author.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. xxx, No. xxx ISSN 2150-8097.
doi:xxxx

rounds. In each round, workers train local models and submit updates to validators, who then verify and aggregate qualified updates to produce a new global model. This paradigm relies on two critical components: *an incentive scheme*, which encourages self-interested participants, including workers and validators, to behave honestly, and *a malicious behavior handling scheme*, which uses consensus mechanisms, such as committee voting [49, 95, 110], to guarantee validity of the final global model. While this paradigm offers a foundation for FMC in Web3, existing BCFL algorithms face critical challenges in efficiency and scalability when applied to FMs:

**C1. Individual computation and memory overhead.** FMs impose prohibitive training and inference costs on both workers and validators. Even moderately sized models such as BERT-Large already exceed the memory capacity of commodity GPUs when full forward–backward computation or multi-update verification is required, as shown in Table 1. As a result, a large fraction of honest participants, particularly those with valuable domain data but constrained by hardware, are excluded from effective participation.

**C2. System-level scalability bottleneck.** Beyond individual resource constraints, the current BCFL paradigm fundamentally fails to scale at the system level. Existing designs require each validator to verify all workers' updates in every round [42, 64, 69], leading to per-validator costs growing linearly with the number of workers (see Table 1). This makes the validator pipeline the dominant bottleneck for large-scale FMs, severely increasing consensus latency and undermining the benefits of engaging more workers.

**C3. Limitations of existing efficiency techniques.** Prior approaches to efficient FM customization, such as LoRA[32], BitFit [114], can reduce worker-side computation or communication, but fail to address validator-side scalability. Conversely, model compression techniques that reduce validation cost, including distillation [30], pruning [48], quantization [29], might sacrifice model quality. Critically, none of these techniques overcomes the linear scaling of verification and consensus in the current BCFL systems.

This paper proposes `DICE`, a novel `D`ecentralized `I`n`CE`ntive-driven BCFL algorithm for efficient, scalable, and secure FMC in Web3. `DICE` completely redesigns the two core steps of previous BCFL algorithms–training and verification & aggregation (VA)–to lower the entry barrier for FMC, enabling a wider range of Web3 participants, including those with hardware constraints, to contribute meaningfully. In the training step, `DICE` adopts a *Dynamic Progressive Learning (DPL)* scheme to reduce the worker-side hardware resource requirements and system-level communication costs by selecting only partial layers of the global FM to train in every round. DPL also dynamically adjusts the layers to be trained, ensuring that all layers necessary to improve the customized FM performance are tuned on workers to maximize FM's adaptation ability. In the VA step, `DICE` employs a *Model Slice Verification (MSV)* scheme, which constructs compact validation models, even smaller than worker-side models, to enable honest but hardware-constrained participants to engage as validators for malicious behavior prevention. The security of the entire FMC process is guaranteed by cross-validator consensus.

To enhance the efficiency and scalability of consensus without compromising security, we introduce a *Tree-structured Committee Consensus (TCC)* mechanism and an *Efficiency-Driven Incentive (EDI)* scheme. TCC decomposes the VA task of each round into tree-structured sub-tasks to enable each validator to execute only a random subset. To ensure security under such partial execution, TCC employs a *selective* voting strategy, focusing voting on root and controversial sub-tasks while directly incorporating the results of non-root, non-controversial sub-tasks into the global consensus. A complete global consensus is established only when (i) sufficient validators have participated in a small, predefined threshold of sub-tasks, and (ii) every sub-task has consensus results. This design ensures that a validator's mandatory workload grows primarily with the number of controversial sub-tasks, rather than scaling linearly with the number of worker updates. To preempt the intentional provocation of conflicts, the complementary EDI scheme imposes substantial penalties on validators whose outputs diverge from the consensus, economically disincentivizing conflict generation. Jointly, TCC and EDI enable `DICE` to achieve a robust, tunable balance between efficiency, scalability, and security, three crucial properties for real-world implementation of FMC in Web3.

The contributions of this paper are outlined as follows:

- **Systematic Problem Analysis and Novel Algorithm.** We systematically identify and analyze the efficiency and scalability challenges that hinder the direct application of existing BCFL paradigms to FMC in Web3. We also propose `DICE`, the first BCFL algorithm specifically designed for FMC in Web3, which significantly lowers FMC's entry barriers, enabling even hardware-limited Web3 participants to contribute effectively. (§3)
- **Novel Technical Designs.** We introduce four novel techniques within `DICE`: DPL for worker-side cost reduction, MSV for validator-side lightweight verification, TCC and EDI to balance efficiency, scalability, and security in cross-validator consensus. (§3.2–3.3)
- **Theoretical Guarantees.** We provide a rigorous theoretical analysis demonstrating that `DICE` achieves optimal per-validator computational complexity in consensus, scaling sub-linearly with the number of workers' updates, while guaranteeing prevention of malicious behaviors with overwhelming probability. (§4)
- **Experimental Validation.** We conduct extensive experiments across both vision and language FMC tasks to evaluate `DICE` consistently outperforms baselines across multiple dimensions: significantly reducing both worker-side and validator-side costs, delivering competitive final model performance, and maintaining robust resistance against malicious actors. These findings collectively confirm `DICE`'s efficiency, effectiveness, and security. (§5)

## 2 PRELIMINARIES

We begin by presenting the participants, threat model, and foundational techniques in prior BCFL systems that `DICE` builds upon. The primary concepts used in this paper are defined as follows.

**Blockchain, Blockchain Network and Consensus chain.** A *blockchain* is a local tamper-resistant database managed by a single computer (called miner). A *blockchain network* is a peer-to-peer network where miners synchronize local blockchains using a consensus algorithm to agree on the same data stored on the blockchain. A synchronized blockchain is called a *consensus chain*.

**Off-blockchain Procedure and On-blockchain Procedure.** An *off-blockchain* procedure is a computation performed by a decentralized algorithm's participant to process its local data. An *on-blockchain* procedure is a computation programmed in the smart

contract [64], written in high-level programming languages such as Solidity [1], and executed on the blockchain network to process data stored in the consensus chain.

## 2.1 Participants and Threat Model

Similar to existing BCFL systems, DICE involves three types of participants: model *requester*, *workers*, and *validators*, as introduced in §1. DICE's model requesters need abundant data held privately by workers to customize large-scale pretrained FMs [12] for superior performance on downstream tasks. They possess powerful hardware to compute on full FMs, but workers and validators might not deploy a full FM due to limited hardware resources.

Workers and validators are trustless and may disrupt FMC via malicious behavior. Existing research in BCFL primarily considered two forms of malicious behavior: (1) *poisoning* [15, 19, 38, 46, 49, 59, 72], an attack aiming to degrade the model performance by committing a poisoned update or corrupted verification/aggregation results. (2) *fraud* [16–18, 43, 49, 104], also known as free-riding [64], an attack to obtain undeserved rewards by submitting a copy of another honest worker's update or another honest validator's VA results; Except for poisoning and fraud, our recent work [49] defined another type of malicious participant, called *failed participants*, who dynamically leave the joined learning task (due to network issues, etc.) then cause learning to get stuck.

DICE adopts a comprehensive threat model encompassing poisoning, fraud, and failed participants, as the foundation of security designs. Malicious participants can launch either form of attack independently, or collude to make their actions more potent and evasive. Each malicious behavior can also be mounted through Sybil attacks [21], where an adversary controls multiple fake identities to amplify its attack's impact. Our threat model is confined to malicious behaviors that directly undermine the final model performance and the integrity of incentive schemes, aligning with mainstream designs in BCFL research [42, 64, 115].

## 2.2 Foundational Techniques

BCFL systems typically rely on a *decentralized iterative learning structure* and a *malicious behavior handling scheme* to ensure the security and effectiveness of federated learning in Web3.

*2.2.1 Decentralized Iterative Learning Structure.* DICE adopts the same learning structure as prior BCFL paradigms, employing a blockchain network to coordinate participants to learn a *global model* by iterative rounds. To initiate a learning task, the model requester creates a smart contract that manages the cryptocurrency for incentivizing participants, the task specifications such as hyperparameters, validation data [85, 91, 110, 113]. Next, workers and validators join the task by registering their unique identifiers, usually their network addresses, on the smart contract. Then, the iterative learning starts. During this process, participants need to exchange data that is too large or expensive to store in the blockchain network. We follow previous studies [14, 49, 110, 111] to use a *secure messaging scheme*, which stores large-scale data such as model parameters in an additional storage system such as IPFS outside of the blockchain network and persists the data file handles in the blockchain network for exchange. Building upon the above learning structure, DICE integrates four innovations (DPL, MSV, TCC, EDI) to improve efficiency and scalability of FMC in Web3 while maintaining model performance and learning security.

*2.2.2 Malicious Behavior Handling.* The prevalent paradigm for malicious behavior prevention in BCFL relies on validators to verify every worker's update, aggregate qualified updates, and reach consensus on verification&aggregation (VA) results. An algorithm is secure against both malicious workers and validators, as long as the consensus VA results are *valid*, i.e., produced by honest validators.

Most BCFL systems employ *consensus algorithms* to filter out poisoned VA results and ensure the validity of final accepted results. These algorithms generally fall into three types [42, 64]: incentive-based [26, 61, 76], Byzantine-fault-tolerance-based [4, 45], and permission-based [35, 90]. Our work is built upon the incentive-based paradigm that employs *economic staking and committee voting* for consensus [49, 95, 110]. This paradigm enables open participation of FMC for anyone willing to contribute data or hardware resources, while ensuring robustness against malicious behavior and offering superior efficiency over alternatives like PoW [79, 115]. It has been proved that if the cryptocurrency that honest validators stake for a BCFL task exceeds $\frac{2}{3}$ of that task's total stake, the probability that any validator succeeds in poisoning attacks is negligible [49].

Though standard consensus algorithms guarantee validity, they typically lack inherent mechanisms to verify the authenticity of the result's authorship, leaving the system vulnerable to fraud attacks by malicious validators. To close this gap, a common solution [49, 54, 87] is to integrate *a cryptographic commitment scheme* into the consensus algorithm. This scheme requires VA results to be submitted in two phases: *committing* and *revealing*. When committing, validators submit *fingerprint* and *commitment* generated by hashing their raw VA results. Only if validators reach consensus on fingerprint, will they reveal the complete raw VA results. It has been proved that, given any probability threshold $\epsilon \in [0, 1)$, as long as the hash length exceeds $\log_{\frac{1}{2}} \epsilon$, the probability that any validator succeeds in fraud attacks is smaller than $\epsilon$ [49].

Building upon the prevalent paradigm of relying on validators for worker verification and on consensus algorithms and cryptographic commitments for validator verification, DICE's malicious behavior handling scheme innovates in two aspects. First, it integrates three novel techniques (MSV, TCC, EDI) to achieve higher efficiency and scalability. Second, it diverges from previous BCFL research that often treats failed participants as an orthogonal issue or implicitly adopts strategies from classical FL [34, 36, 73, 81–83]. DICE classifies failed participants as a particular type of malicious participant, handling them with our previously proposed scheme [49]. This scheme prevents system stall by a dynamically adjusted timeout for each stalling-prone task status, refined via moving averages of the status's actual execution time. Being fully compatible with DICE's novel techniques, this scheme is directly incorporated into DICE.

# 3 THE DICE ALGORITHM.

This section presents our DICE algorithm. §3.1 first provides a high-level overview of the algorithm. Then, §3.2–3.3 illustrate the four novel techniques that DICE utilizes to improve efficiency and scalability while maintaining security and effectiveness of FMC in Web3. Table 2 summarizes the main notations used throughout this paper.

**Table 2: Notation Summary.**

| Symbol | Definition |
|---|---|
| $r, R, \eta$ | global round index, total number of rounds, learning rate |
| $w, v$ | worker's or validator's address on the blockchain network ♠ |
| $\mathbb{W}_{all}, \mathbb{V}_{all}$ | the complete set of workers or validators |
| $\mathbb{W}^r, \mathbb{V}^r$ | the set of active workers or validators in round $r$ ♦ |
| $\theta^r, \theta^r_w$ | global FM parameters and worker $w$'s update in round $r$ |
| $\mathbb{VAR}_v$ | the set of results produced by validator $v$ for a VA task |
| $L_G$ | number of T-layers in global FM |
| $L_E$ | minimum number of T-layers in an emulator |
| $L_w, L_v$ | number of T-layers deployable on worker $w$ or validator $v$ |
| $\mathbb{M}^r_w$ | model slice selected to be trained on worker $w$ in round $r$ |
| $\mathbb{LUS}_j$ | the $j$-th subset of the slice $\cup_{w \in \mathbb{W}^r} \mathbb{M}^r_w$ |
| $\theta^r_{*,\mathbb{LUS}_j}$ | $\mathbb{LUS}_j$'s parameters from any specific or set of worker(s) |
| $N_c, N_t$ | thresholds used in the consensus process |
| $E_{total}$ | total amount of security deposit staked by all validators |
| $E_c$ | expected amount of security deposit selected for a committee |
| $\epsilon_j, \epsilon_c$ | model performance threshold, voting threshold |
| $WR, VR$ | rewards for workers and validators in each round |
| $\lvert * \rvert$ | the number of the elements in a set. |

♠ Each worker/validator is represented by its unique address.
♦ "Active" means having submitted an update or VA results in the current round.

## 3.1 Overview

Fig. 2 presents an overview of DICE, an algorithm designed for FMC tasks, where the pretrained FM comprises an embedding layer, intermediate transformer layers (called T-layers), and an output layer. DICE executes each FMC task by iterative rounds, each containing a *training* step, where workers produce updates of a global FM, and a *VA* step, where validators verify workers' updates and aggregate qualified updates to produce a new global FM. In the training step, we introduce a DPL scheme to enable hardware-constrained workers, especially those having abundant task-specific data, to engage. This innovation divides the training step into two statuses: PREPARATION and TRAINING. In the VA step, we propose MSV, TCC, and EDI to reduce each validator's cost while maintaining the algorithm's security. The TCC mechanism, integrated with a cryptographic commitment scheme, structures the VA step into the VA_COMMIT and VA_REVEAL statuses. Algorithms 1–2 present DICE's core procedures in the four task statuses.

PREPARATION (Fig. 2, orange section). In the first task status, model requester prepares the necessary materials for the subsequent lightweight training and verification (Alg. 1 lines 1–9, Alg. 2 lines 1–4). It applies DPL to select a small slice of global FM for each worker and, to support MSV, may further partition the selected slice into non-overlapping sub-slices for hardware-limited validators (Alg. 1 lines 4–6). For every resulting slice/sub-slice, the T-layers excluded from that slice are then compressed into a lightweight emulator [30] (line 7). After the file handles of emulators and global FM are submitted to the blockchain (lines 8–9), the model requester changes the task status to TRAINING via RequesterOnchainProcedure() (Alg. 2 line 4).

TRAINING (Fig. 2, blue section). During TRAINING, workers cost-effectively produce updates of FMs (Alg. 1 lines 10–14, Alg. 2 lines 5–9). Through DPLtrain() in line 13 of Alg. 1, each worker constructs a compact local model using the emulator and the FM slice selected

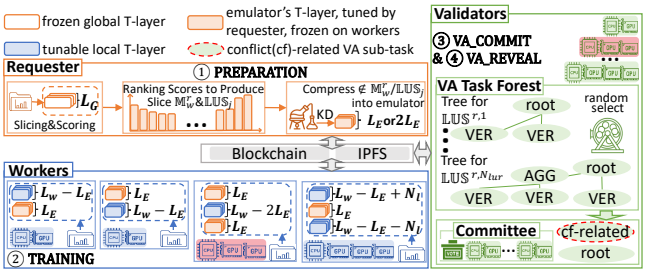



**Figure 2: The overview of DICE.**

by the model requester, then trains the compact model on its private data to produce updates for the selected FM slice. After $|\mathbb{W}^r|$ workers have submitted updates, WorkerOnchainProcedure() changes the task status to VA_COMMIT (Alg. 2 line 9).

VA_COMMIT&VA_REVEAL (Fig. 2, green section). In these two statuses, validators verify and aggregate workers' updates (Alg. 1 lines 15–26, Alg. 2 lines 10–26). Every validator firstly retrieves a VA task forest from smart contract (Alg. 1 line 18). This forest is constructed based on TCC, where each tree node represents a VA sub-task to either verify a worker's update based on MSV or aggregate qualified updates. Each validator randomly selects sub-tasks from the forest to execute, based on its local hardware condition (lines 20–23). Results of selected sub-tasks are submitted via ValidatorOnchainProcedure(), which accepts only fingerprint and commitment of every sub-task's results, not the complete raw results, in the VA_COMMIT status (Alg. 2 line 15). ValidatorOnchainProcedure() checks whether validators reach consensus on the fingerprints of all VA sub-tasks (lines 16–21), and if true, transitions the task status to VA_REVEAL (line 21). Then, validators submit the unexposed raw VA results (Alg. 1 lines 25–26). ValidatorOnchainProcedure() verifies each validator's complete VA results (Alg. 2 lines 23–24) After all validators who participated in VA_COMMIT have revealed their complete VA results, EDI() in line 25 rewards honest participants and punishes malicious validators. Finally, the FMC task proceeds to the next round (line 26).

Below, we detail DPL & MSV for lightweight training&verification (§3.2), and TCC & EDI for efficient cross-validator consensus (§3.3).

## 3.2 Lightweight Training and Verification powered by DPL & MSV

In this section, we introduce *DPL*, a scheme to cost-efficiently train large-scale FMs on workers, and *MSV*, a scheme associated with DPL for cost-effectively verifying each worker's update. DPL aims to enable more workers in FMC and customize all necessary layers of FM on those workers to maximize FM's adaptation ability, while MSV aims to engage more validators, including those more hardware-constrained than workers, to verify updates, increasing the total hardware resources that can be leveraged to accelerate verification. We achieve these objectives through two critical procedures: *Slice Selection* and *Large-to-Compact Model Derivation* performed by the model requester (Alg. 1 lines 4–7).

*3.2.1 Slice Selection.* In this procedure, model requester divides the global FM into several slices, each containing a small number

**Algorithm 1** DICE's off-blockchain procedures.

1: **function** RequesterOffchainProcedure( ) ▷ prepare for §3.2
2: $status, r, r_g \leftarrow$ ReadBlockchain()
3: **if** $status \neq$ PREPARATION or $r \leq r_g$ **then** continue
4: $\{\theta_l^r | l \in [1, L_G]\} \leftarrow$ SplitFMbyLayer()
5: $\{\mathbb{M}_w^r | w \in \mathbb{W}^r\}, \{\mathbb{LUS}_j | j \in [0, N_{lus})\} \leftarrow$ SliceSelection()
6: $\mathbb{M} \leftarrow \{\mathbb{M}_w^r | w \in \mathbb{W}^r\}; \mathbb{LUS} \leftarrow \{\mathbb{LUS}_j | j \in [0, N_{lus})\}$
7: $\{\theta^r_{\notin \mathbb{M}_w^r} | w \in \mathbb{W}^r\}, \{\theta^r_{\notin \mathbb{LUS}_j} | j \in [0, N_{lus})\} \leftarrow$ ModelDerivation()
8: $\mathbb{EM}_w, \mathbb{EM}_v, \mathbb{FM} \leftarrow$ RequesterIPFSStore()
9: RequesterOnchainProcedure($\mathbb{FM}, \mathbb{EM}_w, \mathbb{EM}_v, \mathbb{M}, \mathbb{LUS}$)
10: **function** WorkerOffchainProcedure( ) ▷ §3.2
11: $status, r, r_w \leftarrow$ ReadBlockchain()
12: **if** $status \neq$ TRAINING or $r \leq r_w$ **then** continue
13: $\{\theta^r_{w,\mathbb{LUS}_j} | j \in [0, N_{lus})\} \leftarrow$ DPLtrain() ▷ layers $\notin \mathbb{M}_w^r$ are frozen
14: $\mathbb{LU} \leftarrow$ WorkerIPFSStore(); WorkerOnchainProcedure($\mathbb{LU}$)
15: **function** ValidatorOffchainProcedure( ) ▷ §3.2&§3.3
16: $status \leftarrow$ ReadBlockchain()
17: **if** $status \notin \{$VA_COMMIT, VA_REVEAL$\}$ **then** return
18: $\mathbb{T}_{VA}, \mathbb{T}_v \leftarrow$ ReadBlockchain(); ▷ $\mathbb{T}_v$ stores sub-tasks done by $v$
19: **if** $status =$ VA_COMMIT **then**
20: **while** $status =$VA_COMMIT & $|\mathbb{T}_v| \leq |\mathbb{T}_{VA}|$ **do**
21: $\mathbb{RES}_v^{commit} \leftarrow$ MSVorAGG()
22: ValidatorOnchainProcedure($\mathbb{RES}_v^{commit}$)
23: $status, \mathbb{T}_v \leftarrow$ ReadBlockchain()
24: **else if** $|\mathbb{T}_v| > 0$ **then**
25: $\mathbb{RES}_v^{reveal} \leftarrow$ RevealResPacking()
26: ValidatorOnchainProcedure($\mathbb{RES}_v^{reveal}$)

**Algorithm 2** DICE's on-blockchain procedures.

1: **function** RequesterOnchainProcedure($\mathbb{FM}, \mathbb{EM}_w, \mathbb{EM}_v, \mathbb{M}, \mathbb{LUS}$)
2: **if** $status \neq$ PREPARATION or $r_g \geq r$ **then** return
3: RecordRequesterRes($\mathbb{FM}, \mathbb{EM}_w, \mathbb{EM}_v, \mathbb{M}, \mathbb{LUS}$)
4: $r_g \leftarrow r$; $status \leftarrow$ TRAINING
5: **function** WorkerOnchainProcedure($\mathbb{LU}$)
6: **if** $status \neq$ TRAINING or $r_w \geq r$ **then** return
7: $r_w \leftarrow r$; $\mathbb{AW}^r \leftarrow$ RecordRequesterRes($\mathbb{LU}$);
8: **if** $\mathbb{AW}^r = \mathbb{W}^r$ **then** ▷ sufficient active workers
9: CreateVATaskTree(); $status \leftarrow$ VA_COMMIT ▷ §3.3
10: **function** ValidatorOnchainProcedure($\mathbb{RES}_v$)
11: **if** $status \notin \{$VA_COMMIT, VA_REVEAL$\}$ **then** return
12: **if** $status =$VA_COMMIT **then** OnchainCommit($\mathbb{RES}_v$)
13: **else** OnchainReveal($\mathbb{RES}_v$)
14: **function** OnchainCommit($\mathbb{RES}_v$) ▷ §3.3
15: $tid, \mathbb{VAR}_v^{commit}, FP_v, CC_v, \pi_v, hash_v \leftarrow \mathbb{RES}_v$
16: **if** not Record($tid, \mathbb{VAR}_v^{commit}, FP_v, CC_v, \pi_v, hash_v$) **then** return
17: **if** NeedCommitteeVoting($tid$) **then**
18: Voting($tid, \mathbb{VAR}_v^{commit}, FP_v, CC_v, \pi_v, hash_v$)
19: **else** CheckConflict($tid, \mathbb{VAR}_v^{commit}, FP_v, CC_v, \pi_v, hash_v$)
20: **if** $|\{v | v \in \mathbb{V}_{all}, |\mathbb{T}_v \setminus \mathbb{CR}| \geq \frac{|\mathbb{T}_{VA}| - |\mathbb{CR}|}{|\mathbb{T}_{VA}|} N_t\}| < N_c$ **then** return
21: **if** $|RES^r| \geq \mathbb{T}_{VA}$ **then** $status \leftarrow$ VA_REVEAL
22: **function** OnchainReveal($\mathbb{RES}_v$) ▷ §3.3
23: $\{< tid, \mathbb{VAR}_v^{reveal} > | tid \in \mathbb{T}_v\} \leftarrow \mathbb{RES}_v$
24: QualificationCheck($\{< tid, \mathbb{VAR}_v^{reveal} > | tid \in \mathbb{T}_v\}$)
25: **if** all($\{|\mathbb{T}_v| = 0 | v \in \mathbb{V}_{all}\}$) **then** EDI()
26: OnchainPreparation(); $status \leftarrow$ PREPARATION

of T-layers, then cyclically selects a small slice of global FM to train on workers based on each slice's importance score and executes further-slicing for validators when necessary (Alg. 1 line 4).

For each worker $w$, the model requester performs slice selection based on our DPL scheme. The global FM is divided into a set of slices $\mathbb{M}_{w,i}$ defined in Eq. 1, where $N_s' = L_G - L_w$, $N_s = L_G - L_E - 1$, and the numbers in $\mathbb{M}_{w,i}$ denote the indices of global T-layers.

$$\mathbb{M}_{w,i} = \begin{cases} \{1, \cdots, L_w - L_E\} & \text{if } i = 0 \text{ (shallow)} \\ \{1 + L_E + i, \cdots, L_w - L_E + i\} & \text{if } i \in [1, N_s') \text{ (middle)} \\ \{L_G - (L_w - L_E - 1), \cdots, L_G\} & \text{if } i = N_s' \text{ (top)} \\ \{1, \cdots, L_w - L_E - (i - N_s'), & \text{if } i \in (N_s', N_s] \\ \ L_G - (i - N_s' - 1), \cdots, L_G\} & \text{(combined)} \end{cases} \quad (1)$$

To accelerate customization of all necessary layers, our DPL scheme defines an importance score (Eq. 2) for each slice $\mathbb{M}_{w,i}$. The score incorporates two key metrics: (1) the performance $s^r_{\mathbb{M}_{w,i}}$ of the global FM after $\mathbb{M}_{w,i}$ is tuned, and (2) the parameter divergence $div^r_{\mathbb{M}_{w,i}} = 1 - cos_sim(\theta^r_{\mathbb{M}_{w,i}}, \hat{\theta}^r_{\mathbb{M}_{w,i}})$, where $\theta^r_{\mathbb{M}_{w,i}}, \hat{\theta}^r_{\mathbb{M}_{w,i}}$ denote the slice parameters before and after tuning, respectively.

$$Score_{w,i} = \frac{s^r_{\mathbb{M}_{w,i}}}{\max(\{s^r_{\mathbb{M}_{w,i}} | i \in [0, N_s]\})} + \frac{div^r_{\mathbb{M}_{w,i}}}{\max(\{div^r_{\mathbb{M}_{w,i}} | i \in [0, N_s]\})} \quad (2)$$

Under this design, slices that yield greater performance improvement and exhibit more substantial parameter updates are prioritized for training. The model requester computes the slice's importance score, selects $\mathbb{M}_{w,i}$ with the highest score and combines it with the output layer to form the final slice $\mathbb{M}_w^r$ for training on worker $w$.

At the end of slice selection, the model requester performs *further slicing* based on our MSV scheme to enable more validators to participate in verifying worker updates. It separates the complete set of worker-updated slices into non-overlapping sub-slices $\{\mathbb{LUS}_j | j \in [0, N_{lus})\}$, if any worker's update cannot be deployed together with an emulator in a lightweight validation model containing $L_{vm}$ T-layers. Here, $L_{vm}$ is determined by the minimum number of T-layers that can be deployed on hardware-constrained validators; $N_{lus} = \lceil \frac{|\cup_{w \in \mathbb{W}^r} \mathbb{M}_w^r \setminus \{output\}|}{L_{vm} - 2L_E} \rceil$, where $\mathbb{W}^r$ is the set of active workers in round $r$. If the above condition does not hold, $N_{lus} = 1$ and $\mathbb{LUS}_j = \cup_{w \in \mathbb{W}^r} \mathbb{M}_w^r$. For every slice $\mathbb{M}_w^r$ and $\mathbb{LUS}_j$, the model requester compresses the global T-layers excluded from that slice into the emulator in a compact model deployable on any worker/validator through ModelDerivation() in line 7 of Alg. 1.

*3.2.2 Large-to-Compact Model Derivation.* In this procedure, model requester derives compact models from the global FM for both workers and validators (Alg. 1 line 7). Each compact model retains the same embedding and output layers as the global FM but contains a reduced number of T-layers. These compact models are designed to allow workers to efficiently generate updates for $\mathbb{M}_w^r$, and to enable validators to verify each update at a manageable cost. Motivated by offsite tuning [99], we insert a lightweight *emulator* into each compact model to ensure that each worker's update and the corresponding verification results closely approximate those obtained by training or verifying against the full FM, thereby achieving model utility comparable to full FM customization. As presented in Fig. 3, the emulator is constituted by either $L_E$ or $2L_E$ T-layers, with $L_E$

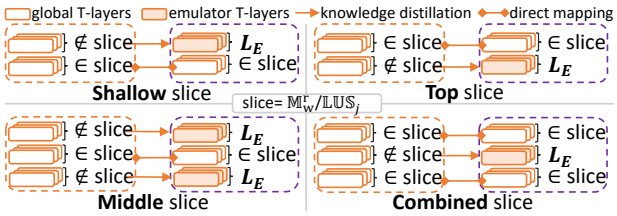



**Figure 3: Generate emulators for workers and validators to build compact local and validation models, respectively.**

**Figure 4: Four types of validation model built from $\theta^r_{*,\mathbb{LUS}_j}$.**

typically set to one and $2L_E$ T-layers adopted only when $\mathbb{M}^r_w$ or $\mathbb{LUS}_j$ is a middle slice. The emulators for worker $w$ and validator $v$ are produced by compressing the global T-layers not in $\mathbb{M}^r_w$ and $\mathbb{LUS}_j$, respectively, through knowledge distillation (KD)[30, 99].

*3.2.3 Lightweight Training and Verification.* Using an emulator, the worker and validator construct a compact local and validation model for training and verification, respectively. Each worker $w$ executes DPLtrain() in line 13 of Alg. 1 to produce $\mathbb{M}^r_w$'s update and divide the update into $\{\theta^r_{w,\mathbb{LUS}_j} | j \in [0, N_{lus})\}$. Each validator $v$ executes MSVorAGG() in line 21 of Alg. 1 to verify and aggregate $\theta^r_{w,\mathbb{LUS}_j}$. An update $\theta^r_{w,\mathbb{LUS}_j}$ will be considered valid, only if it passes both fraud and poisoning checks, with at least one layer whose norm diverges from the historical norms of updates for that layer, and the validation model built upon that update (as shown in Fig. 4) achieving better performance than a threshold $\epsilon_j$ dynamically adjusted by the model requester.

*3.2.4 Discussion.* We finally conclude this subsection with a discussion on the design rationales of our proposed schemes, particularly regarding the optimality of DPL's layer selection and the robustness of MSV against potential verification bias.

The design of our DPL scheme is fundamentally oriented towards optimizing information gain under the hard constraints of heterogeneous hardware across workers. The slice importance score in Eq. 2 directly quantifies the expected utility of training a specific set of layers, combining both the potential for performance improvement and the magnitude of learned information. By selecting the slice that maximizes this score, DPL provides a principled, near-optimal solution to the problem of which part of FM should be customized at any given round, rather than a purely heuristic scheme.

To accommodate different resource availability, we introduce two operational modes for DPL: (1) *Unified* (U-DPL), where the same slice is selected for all workers by using a unified parameter $L_{wm} = \min(\{L_w | w \in \mathbb{W}_{all}\})$ to replace every worker's specific parameter $L_w$ in the slice selection step introduced in §3.2, ensuring consistency; (2) *Personalized* (P-DPL), where slices are selected per worker to maximize the utilization of available hardware, as described in §3.2. In both modes, we adopt a periodic design to prevent unnecessary computational overhead, where the requester re-computes slice importance scores only when the $c$-round average performance of the global FM declines, ensuring that the slice training schedule adapts only when the model's learning trajectory stagnates. The model requester, though typically the hardware-unlimited entities, can also adopt parameter-efficient fine-tuning techniques, such as LoRA [32], BitFit [114], during the importance score computation to further reduce costs, a practical optimization we leave for future exploration.

`DICE` runs in U-DPL mode by default to accelerate the model requester's procedures, as U-DPL needs the slice selection step to be executed only once for all workers in any round. P-DPL will be activated, only if there exist hardware-rich workers (with $L_w > L_{wm}$) whose updates consistently achieve a performance score higher than $\frac{L_w}{L_{wm}} \cdot s_{wm}$, where $s_{wm}$ is the maximum performance score that hardware-constrained workers (i.e., with $L_w = L_{wm}$) have ever achieved in all past rounds. In this case, the model requester selects a larger-sized slice for those workers using their specific $L_w$, while remaining to use $L_{wm}$ in the slice selection for the other workers. Once the above condition is not satisfied, `DICE` will switch back to U-DPL mode. Such an adaptive mode-switching mechanism is motivated by the empirical finding that the benefit of personalizing the sizes of local tunable parameters is contingent on the alignment between hardware capacity and local training quality. If hardware-rich workers cannot produce well-performing model updates, assigning a larger slice for them could lead to overfitting on those workers, reducing the generality of the global T-layers that hardware-constrained workers do not update, thereby compromising the final global FM performance. Therefore, `DICE` chooses to activate P-DPL only when capable workers demonstrate consistently superior local performance exceeding a dynamic threshold.

Our MSV scheme is designed with a deliberate trade-off to enable verification on resource-limited devices while mitigating a potential verification bias that could arise from using a distilled emulator. It employs a dynamic threshold $\epsilon_j$, adjusted each round based on the global model's performance trend, to automatically correct for any systematic under- or over-estimation of update quality. This dynamic adjustment also ensures that MSV remains a flexible framework, not rigidly tied to specific attack patterns. As new adversarial strategies emerge, the defense can be updated by refining the adjustment logic for $\epsilon_j$, making MSV a durable and adaptable solution for decentralized FMC verification.

While DPL and MSV reduce the costs of producing and verifying an individual update, they do not address the efficiency and scalability challenge arising from the requirement on every validator to verify all updates to ensure the validity of final customized FMs [49, 95, 110]. To this end, we introduce two novel techniques in the next section.

## 3.3 Balancing Efficiency, Scalability, and Security by TCC & EDI

This section introduces TCC and EDI, two integrated techniques to minimize validator-side computational complexity and thereby maximize system-wide scalability while ensuring security. TCC

overcomes the linear dependency between each validator's workload and the number of worker updates by decomposing the monolithic VA task into tree-structured sub-tasks and employing a selective committee voting strategy that invokes voting only for root sub-tasks and sub-tasks that become controversial. However, TCC's efficiency gain might be compromised if validators deliberately trigger conflicts to increase consensus latency. EDI complements TCC by imposing economic disincentives against such behavior, restoring the efficiency-scalability-security trade-off. Below, §3.3.1 details how TCC generates consensus, §3.3.2 presents how EDI works.

*3.3.1 Consensus Generation.* TCC generates consensus over the entire VA task by four critical steps: *Task Decomposition, Result Committing, Selective Committee Voting*, and *Result Revealing*.

*(1) Task Decomposition.* In this step, a monolithic VA task is decomposed into $N_{VA}$ sub-tasks in a VA task forest via `CreateVATaskTree()` in line 9 of Alg. 2, where $N_{VA} \leq (2|\mathbb{W}^r| - 1)N_{lus}$. The forest comprises several task trees, as shown in Fig. 2. In $Tree_j$, $j \in [0, N_{lus})$, the leaf nodes represent EVA sub-tasks, each verifying a worker's update of $\mathbb{LUS}_j$. The non-leaf nodes, conversely, represent AGG sub-tasks, where each validator first aggregates qualified updates from two child nodes, then computes the contribution of each child node's update to the aggregated update. Executing each non-leaf sub-task has a strong dependency on the results of its two child sub-tasks. For example, an AGG sub-task requires the performance scores computed in its child sub-tasks to determine each update's weight and contribution in aggregation. After the VA task forest is constructed on the blockchain, result committing starts.

*(2) Result Committing.* Each validator selects sub-tasks from the forest, computes their results, and submits fingerprints and commitments of the produced results to the blockchain based on a cryptographic commitment scheme to prevent fraud, via ValidatorOffchainProcedure() in lines 19–23 of Alg. 1.

For each selected sub-task, validator $v$ produces the result set $\mathbb{VAR}_v$ based on the sub-task type. If the sub-task is EVA, the validator $v$ first verifies a worker's update for $\mathbb{LUS}_j$ as illustrated in §3.2.3. If the update is qualified, the validator computes that update's weight in the final aggregation results of slice $\mathbb{LUS}_j$. To achieve faster convergence of FMC, we assign a larger weight $\alpha_{w,\mathbb{LUS}_j}$ to $\theta^r_{w,\mathbb{LUS}_j}$ if (1) the validation model built from $\theta^r_{w,\mathbb{LUS}_j}$ achieves a higher performance $s_{w,\mathbb{LUS}_j}$, and (2) $\theta^r_{w,\mathbb{LUS}_j}$ diverges more significantly from its corresponding global FM parameters $\theta^r_{\mathbb{LUS}_j}$. Therefore, we define $\alpha_{w,\mathbb{LUS}_j}$ by Eq. 3, where $\lambda$ is a constant for value scaling. Finally, the validator produces the result set $\mathbb{VAR}_v$ containing $\alpha_{w,\mathbb{LUS}_j}$ and the final outputs $V(\theta^r_{w,\mathbb{LUS}_j}, \mathbb{D}_{val})$ from the validation model built upon $\theta^r_{w,\mathbb{LUS}_j}$ and evaluated on $\mathbb{D}_{val}$.

$$\alpha_{w,\mathbb{LUS}_j} = \exp(\lambda \cdot s_{w,\mathbb{LUS}_j})[1 + \tanh(\lambda||\theta^r_{w,\mathbb{LUS}_j} - \theta^r_{\mathbb{LUS}_j}||)] \quad (3)$$

In each AGG sub-task, the validator $v$ produces the result set by two key steps: aggregation and contribution computation. The aggregation is designed to ensure that the final result at the root node of $Tree_j$ yields a weighted average of the individual updates $\theta^r_{w,\mathbb{LUS}_j}$ from all workers $w \in \mathbb{W}_j$ who submitted valid updates for the slice $\mathbb{LUS}_j$. To achieve this, we first define a recursive weighting mechanism. For a node with child nodes $c_1$ and $c_2$, the weighting factor $\frac{\beta_{\mathbb{W}_c,\mathbb{LUS}_j}}{\tau_{c_1,c_2}}$ is defined by Eq. 4–5, where $c \in \{c1, c2\}$, $\mathbb{W}_c$ is the set of workers whose updates are aggregated in child node $c$, and $\mathbb{W} = \mathbb{W}_{c1} \cup \mathbb{W}_{c2}$.

$$\beta_{\mathbb{W}_c,\mathbb{LUS}_j} = \begin{cases} \alpha_{w,\mathbb{LUS}_j}, \text{ where } w \in \mathbb{W}_c & \text{if } |\mathbb{W}_c| = 1 \\ 1 & otherwise \end{cases} \quad (4)$$

$$\tau_{c1,c2} = \begin{cases} \frac{1}{\sum_{w \in \mathbb{W}_j} \alpha_{w,\mathbb{LUS}_j}} & \text{if } |\mathbb{W}_{c1} \cup \mathbb{W}_{c2}| = |\mathbb{W}_j| \\ 1 & otherwise \end{cases} \quad (5)$$

Using this weighting factor, the validator combines the qualified updates received from its two child nodes in the aggregation step via Eq. 6, where $\theta^r_{\mathbb{W}_c,\mathbb{LUS}_j}$ is the update from child node $c$, being a worker's update if $\mathbb{W}_c$ is a singleton or being a partial aggregated update otherwise.

$$\theta^r_{\mathbb{W},\mathbb{LUS}_j} = \sum_{c \in \{c1,c2\}} \frac{\beta_{\mathbb{W}_c,\mathbb{LUS}_j}}{\tau_{c1,c2}} \theta^r_{\mathbb{W}_c,\mathbb{LUS}_j}. \quad (6)$$

After aggregation, the validator quantifies each child node $c$'s contribution to the parent aggregation result using marginal contribution, as defined in Eq. 7, where $s_{\mathbb{W},\mathbb{LUS}_j}$ denotes the performance of the validation model built from the full aggregation result $\theta^r_{\mathbb{W},\mathbb{LUS}_j}$, while $s_{\mathbb{W}\setminus\mathbb{W}_c,\mathbb{LUS}_j}$ denotes the performance when the aggregation excludes the updates from child node $c$. This approach is inspired by the Shapley value [50, 51] that ensures fair reward distribution. The $\Delta$ values will be used in the smart contract to compute the contribution of each worker $w$ by Eq. 8 at the end of each learning round.

$$\Delta_{\mathbb{W}_c,\mathbb{LUS}_j} = s_{\mathbb{W},\mathbb{LUS}_j} - s_{\mathbb{W}\setminus\mathbb{W}_c,\mathbb{LUS}_j} \quad (7)$$

$$Contri_w = \max[0, \sum_{j=0}^{N_{lus}-1} AVG(\{\Delta_{\mathbb{W}_c,\mathbb{LUS}_j} | w \in \mathbb{W}_c\})] \quad (8)$$

When aggregation and contribution computation are both done, the validator generates its result set $\mathbb{VAR}_v$, including $H(\theta^r_{\mathbb{W},\mathbb{LUS}_j})$, $V(\theta^r_{\mathbb{W},\mathbb{LUS}_j}, \mathbb{D}_{val})$, $s_{\mathbb{W},\mathbb{LUS}_j}$, $\Delta_{\mathbb{W}_{c1},\mathbb{LUS}_j}$, and $\Delta_{\mathbb{W}_{c2},\mathbb{LUS}_j}$.

To prevent fraud attacks, each validator submits fingerprints and commitments of its produced results, rather than the raw result set $\mathbb{VAR}_v$, to the blockchain based on a cryptographic commitment scheme. Based on conventional commitment schemes [49], a fingerprint value and its commitment are computed as $FP_v = H(\mathbb{VAR}_v)$, $CC_v = H(v||\mathbb{VAR}_v)$, respectively. Furthermore, TCC extends conventional schemes to include a subset of raw results $\mathbb{VAR}_v^{commit}$ as part of the fingerprint. This design enables a pipelined verification flow: validators executing each non-leaf sub-task can directly use the committed subsets from their child-node results, eliminating the need to re-execute those child tasks, which reduces individual workload and decreases global consensus latency across the VA task forest.

For any given sub-task, honest validators will submit the *same* valid fingerprint, while malicious ones may submit invalid versions. To check validity and produce consensus fingerprints, each validator $v$'s submission is fed to `OnchainCommit()` in lines 16–21 of Alg. 2 via `ValidatorOnchainProcedure()` in line 12. For a non-leaf sub-task submission from validator $v$, it first checks if the blockchain records $v$'s self-produced results for all descendants sub-tasks of this non-leaf node. If $v$ lacks self-produced results for any descendant, the procedure then checks whether the blockchain already contains consensus results for those specific descendants produced

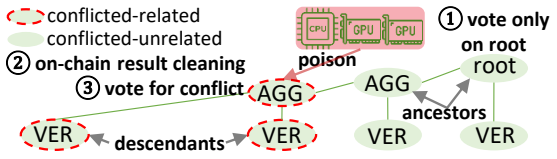



**Figure 5: Selective committee voting.**

by other validators. If neither condition is met, $v$'s submission is discarded (line 16). This ensures honest validators produce valid results for non-leaf tasks without redundant re-execution of that sub-task's descendants. Submissions for all leaf sub-tasks, as well as those passing the above check, proceed to selective committee voting for validity determination.

*(3) Selective Committee Voting.* Fig. 5 depicts TCC's selective voting strategy, implemented by lines 17–19 in `OnchainCommit()`. Committees are formed only for root sub-tasks at the start of consensus process (Fig. 5①). Additional committees are formed dynamically only when a sub-task becomes controversial (Fig. 5③). Committee formation requires each validator to stake at least $E_{\min}$ units of cryptocurrency as a security deposit upon joining an FMC task. Each committee, selected randomly from all validators with probability proportional to their stakes, votes to determine the consensus fingerprint of its assigned sub-task.

`Voting()` in line 18 of Alg. 2 is triggered only if the sub-task is root or controversial. This function first verifies the validator $v$'s committee membership and calculates $v$'s votes using its submitted proof $\pi_v$ and hash value $hash_v$. For details of this verification process, please refer to our previous paper [49]. A validator $v$'s votes are counted only if it meets specific requirements: (i) For a root non-controversial sub-task, $v$ must have committed to at least $\frac{N_{VA}-N_{CF}}{N_{VA}} N_t$ *conflict-unrelated* sub-tasks, where $N_{CF}$ is the number of *conflict-related* sub-tasks including controversial sub-tasks and their descendants; (ii) For a controversial sub-task, $v$ must have committed to all descendant sub-tasks of that controversial task. These requirements ensure that (i) consensus on all conflict-unrelated tasks is produced by honest validators (Theorem 4, §4.2), and (ii) all controversial tasks receive correct inputs from their descendants. If $v$'s votes are eligible, the function then casts them toward the candidate fingerprint that matches $v$'s committed fingerprint. The consensus fingerprint for a root or controversial sub-task is the highest-voted fingerprint that receives more than $E_c \cdot \epsilon_c$ votes.

For the non-root, non-controversial sub-task where voting is unnecessary, `CheckConflict()` in line 19 of Alg. 2 is invoked. If no conflict is detected, the received fingerprint is directly adopted as that sub-task's consensus. Otherwise, that sub-task is marked controversial. It and its descendants become conflict-related, triggering a two-step resolution. This procedure first removes all previous results of that sub-task and its ancestors from the blockchain (Fig. 5②), preventing that sub-task from compromising the validity of its ancestors' results. Then, a new committee is randomly selected to vote on this sub-task's consensus fingerprint (Fig. 5③).

When (1) $N_c$ or more validators have committed to $\frac{N_{VA}-N_{CF}}{N_{VA}} N_t$ conflict-unrelated sub-tasks, and (2) every VA sub-task has a consensus fingerprint, a global consensus is established and task status transitions to `VA_REVEAL` (lines 20–21 of Alg. 2).

*(4) Result Revealing.* During `VA_REVEAL`, each validator $v$ reveals the unexposed raw VA results $\mathbb{VAR}_v^{\text{reveal}} = \mathbb{VAR}_v \setminus \mathbb{VAR}_v^{\text{commit}}$ for each sub-task that it previously committed to (lines 25–26, Alg. 1). Those results are fed into `OnchainReveal()` in lines 22–26 of Alg. 2 via `ValidatorOnchainProcedure()` in line 13 to check whether all following conditions are satisfied: (i) $H(v \parallel \mathbb{VAR}_v^{\text{commit}} \cup \mathbb{VAR}_v^{\text{reveal}}) = CC_v$, (ii) $H(\mathbb{VAR}_v^{\text{commit}} \cup \mathbb{VAR}_v^{\text{reveal}}) = FP_{\text{agreed}}$, (iii) $\mathbb{VAR}_v^{\text{commit}} = \mathbb{VAR}_{\text{agreed}}^{\text{commit}}$. Here, $FP_{\text{agreed}}$ and $\mathbb{VAR}_{\text{agreed}}^{\text{commit}}$ are the consensus results for the sub-task to which $\mathbb{VAR}_v^{\text{reveal}}$ pertains. These checks prevent validator fraud. Any attempt to reuse another validator $v$'s $CC_v$ will fail the first condition, as $CC_v$ is bound to $v$'s identity. Moreover, during `VA_COMMIT`, no validator can compute a commitment that binds to its own identity without the knowledge of full $\mathbb{VAR}_v$. Thus, only honest validators who actually computed $\mathbb{VAR}_v$ can satisfy all three conditions simultaneously. The formal security guarantee of this commitment scheme is provided in Theorem 3 (§4.2). The revealing phase ends when all validators that participated in committing have revealed their full results (line 25).

### 3.3.2 *Reward&Punish.*

At the end of `VA_REVEAL`, our EDI scheme allocates rewards and penalties to workers and validators (Alg. 1 line 25), incentivizing honest behavior and encouraging greater hardware resource contributions to accelerate FMC in `DICE`.

Rather than replacing existing incentive mechanisms, our EDI scheme augments them with workload-sensitive coefficients that proportionally amplify cryptocurrency rewards for: (i) honest workers who generate higher-quality updates across more model parameters, and (ii) honest validators who correctly complete more VA sub-tasks. Formally, the worker's and validator's workload-sensitive coefficients are computed by Eq. 9–10 and their corresponding payoff, denoted by $R_w$ and $R_v$, is computed via Eq. 11. Note that $Final_w$ or $Final_v$ will be adjusted to zero if worker $w$ or validator $v$ is detected as a malicious participant. This design ensures that participants who contribute to system efficiency more substantially receive proportionally larger rewards while preserving the fairness guarantees of the underlying incentive mechanism.

$$Final_w = \Phi_w \cdot (1 + \gamma_w \frac{\sum_{j=0}^{N_{lus}-1} Contri_{w,j}}{\sum_{w' \in \mathbb{W}^r} \sum_{j=0}^{N_{lus}-1} Contri_{w',j}}), \tag{9}$$

$$Final_v = \Phi_v \cdot (1 + \gamma_v \frac{N_{v,t}}{\sum_{v' \in \mathbb{V}^r} N_{v',t}}), \tag{10}$$

$$R_w = \frac{Final_w}{\sum_{w' \in \mathbb{W}^r} Final_{w'}} \cdot WR; \quad R_v = \frac{Final_v}{\sum_{v' \in \mathbb{V}^r} Final_{v'}} \cdot VR, \tag{11}$$

where $\Phi_w$, $\Phi_v$ are base rewards for worker $w$ and validator $v$ computed via prior incentive mechanisms [109], $\gamma_w, \gamma_v \in (0, 1]$ are the tunable incentive amplification parameters, $Contri_{w,j}$ is worker $w$'s contribution score to the $j$-th model slice, $N_{v,t}$ is the number of VA sub-tasks that the validator $v$ correctly completed, $WR$ and $VR$ are per-round total rewards for workers and validators, respectively.

While the above design encourages productivity, it might incentivize validators to deliberately trigger conflicts to buy time to complete more VA sub-tasks for greater reward, thereby increasing consensus latency. To preempt this, EDI punishes validators whose submission conflicts with the final consensus, by burning a portion of their staked deposit. Eq. 12 computes the amount burnt for each incorrect submission in a round $r$ based on the maximum rewards

that a validator can receive in this round, where $\Phi_{vmin}, \Phi_{vmax}$ are the minimum and maximum of the base rewards for an individual validator, respectively. We use this sufficiently large penalty to deter validators from submitting controversial results, preemptively eliminating the latency in conflict resolution.

$$Punish = \frac{1}{1 + \frac{\Phi_{vmin}(N_c-1)(N_{VA}+N_t(N_c-1)+\gamma_v N_t)}{\Phi_{vmax}(N_{VA}+N_t(N_c-1)+\gamma_v N_{VA})}} \cdot VR \tag{12}$$

*3.3.3 Discussion.* We conclude with a discussion on the design principles that enable TCC and EDI to achieve their claimed balance between efficiency, scalability, and security. TCC's core insight is that global consensus does not require every validator to verify every update. By structuring the VA task as a forest and employing selective committee voting, TCC ensures that honest validators need only execute a random subset of sub-tasks while the protocol still produces a globally consistent consensus. This shifts the scalability bottleneck from individual hardware capabilities to the collective power of the validator set, enabling the system to scale with the number of participants rather than being limited by individual hardware constraints. However, a standalone TCC might degrade toward the linear-complexity methods it was designed to replace, if malicious validators trigger too many controversial VA sub-tasks. EDI's punitive mechanism, burning a sufficiently large amount of the offending validator's stake, is calibrated to make such attacks economically irrational. The penalty in Eq. 12 exceeds any potential gain from malicious behavior, ensuring that rational validators abstain from provoking conflicts. With EDI disincentivizing conflicts, we will show in §4 that, by setting $N_t$, $N_c$, $E_c$, $\epsilon_c$ properly, TCC can minimize each validator's computational complexity in achieving global consensus while maintaining security by preventing malicious behaviors with high probability.

# 4 THEORETICAL ANALYSIS

In this section, we analyze DICE's efficiency, scalability, and security in §4.1–4.2, then discuss in §4.3 a fundamental trade-off revealed by these analyses, i.e., improving efficiency and scalability increases the number of validators required for security.

## 4.1 Efficiency & Scalability

This section analyzes the efficiency of our DICE algorithm in terms of the per-worker and per-validator computational complexity in each learning round. A lower complexity is expected because it enables more honest Web3 participants to contribute to FMC tasks even if they are hardware-constrained. Furthermore, since all workers (and similarly, all validators) perform their computations in parallel, the overall task execution time is dominated by the computational complexity of each individual participant within each role. Therefore, for a given FMC task with a fixed set of participants, reducing the per-worker and per-validator complexities directly lowers the total time to complete each round of the task.

For ease of analysis, we suppose that (1) the size of global FM is $S_{FM}$; (2) the minimum size of the local models and validation models in DICE is $\gamma_{LM} S_{FM}$ and $\gamma_{VM} S_{FM}$ respectively (where $0 < \gamma_{VM} \leq \gamma_{LM} < 1$). Beyond DICE, we also analyze existing BCFL paradigms [46, 49, 110] for comparison.

**Theorem 1.** *In each round $r$ of existing BCFL paradigms, the worker's and validator's minimum computational complexity is $O(S_{FM})$ and $O(|\mathbb{W}^r| \cdot S_{FM})$, respectively.*

**Proof.** In existing BCFL paradigms, the model locally trained by any worker has exactly the same model structure as the full global model. Therefore, the computational complexity of all workers is the same $O(S_{FM})$. Given that $|\mathbb{W}^r|$ workers produce updates in round $r$ and the size of every worker's update is $S_{FM}$, each validator must correctly verify all updates if it expects to receive rewards from existing BCFL paradigms. A worker's update is typically verified by testing the model whose size is the same as the given update. Therefore, the minimum computational complexity of the validators in the existing BCFL paradigm is $O(|\mathbb{W}^r| \cdot S_{FM})$.

**Theorem 2.** *In every round $r$ of* DICE*, the worker's and validator's minimum computational complexity is $O(\gamma_{LM} \cdot S_{FM})$ and $O(N_t \cdot \gamma_{VM} \cdot S_{FM})$, respectively.*

**Proof.** In DICE, every worker trains a compact local model which has a reduced number of T-layers compared to the global FM. Therefore, the worker's minimum complexity is linear in the minimum size of local models. Given updates produced by workers $w \in \mathbb{W}^r$, DICE's consensus algorithm divides the task of verifying and aggregating those updates into $N_{VA}$ sub-tasks. If a validator expects to obtain rewards, it needs to correctly produce VA results for at least $\frac{N_{VA}-N_{CF}}{N_{VA}} N_t$ conflict-unrelated sub-tasks for the generation of a valid consensus that moves the FMC task forward. Since $\frac{N_{VA}-N_{CF}}{N_{VA}} N_t \leq N_t$ and the computational cost of each VA sub-task is dominated by the cost of computing the performance score of a validation model (whose size is $\gamma_{VM} \cdot S_{FM}$) to either verify a worker's update or measure the update's contribution, the validator's minimum computational complexity is determined by $N_t$ and $\gamma_{VM} \cdot S_{FM}$.

These theorems indicate that, compared to existing BCFL paradigms, DICE can achieve much lower computational complexities on both workers and validators, thus having a greater potential to enable more honest Web3 participants to contribute to FMC, when $N_t$ is set by a *sublinear* function of $|\mathbb{W}^r|$ (e.g., $\sqrt{|\mathbb{W}^r|}$, $\log |\mathbb{W}^r|$, or even a constant). Setting $N_t$ to $O(1)$ yields optimal per-validator efficiency and system-wide scalability, since it enables the global consensus to be established with constant computational complexity per-validator, independent of the number of worker updates. However, Theorem 4 in the next section will show that unilaterally minimizing $N_t$ (to pursue optimal efficiency and scalability) can lead to an increase in $N_c$ (requiring more validators), necessitating a trade-off between individual validator efficiency and practical feasibility of recruiting sufficient validators, which we will discuss in §4.3.

## 4.2 Security

An algorithm is secure if and only if the algorithm's final results are produced by honest participants who strictly follow the algorithm's logic [49]. DICE relies on honest validators to reject illegal updates submitted by malicious workers and to aggregate only qualified updates to update global FM. Thus, DICE is secure as long as the consensus VA results of each round are generated by honest validators. This security relies on three critical preconditions.

**Assumption 1. Hash Collision Resistance.** Given any probability threshold $\epsilon_p \in (0, 1]$, there exists a cryptographic hash function $H(\cdot)$ and an input parameter domain satisfying $\min(hlen, \log_2 N_{input}) > \log_{\frac{1}{2}} \epsilon_p$, where $hlen$ is the bit-length of $H$'s output, and $N_{input}$ is the cardinality of the input value domain.

**Assumption 2. Sufficient Validators&Stake and Honest Majority.** *The total number of validators and the total security deposit staked by all validators are both sufficiently large, i.e.,* $|\mathbb{V}_{all}| > 2\lceil \log_{[1-\frac{N_t}{(2|\mathbb{W}^r|-1)N_{lus}}]} \epsilon_p \rceil$, $E_{total} >> E_c$. *The ratio of deposit staked by honest validators to* $E_{total}$, *denoted by h, is greater than* $\frac{2}{3}$.

**Assumption 3. Bounded Stake Inequality.** *Let* $E_{min}$, $E_{avg}$ *be the minimum and average stake amounts among all validators, respectively. We assume* $\frac{E_{avg}}{E_{min}} \leq \frac{3}{2}$.

Under these assumptions, we have Theorems 3–5.

**Theorem 3.** *Under Assumption 1, given any small threshold* $\epsilon_p \in (0, 1)$, *the probability that any malicious validator* $v'$ *succeeds in fraud attacks, i.e., passing* `QualificationCheck()` *via a* $CC_{v'}$ *value that binds to its own identity and computed in the* `VA_COMMIT` *status using public results plagiarized from one or more honest validators, is smaller than* $\epsilon_p$.

**Proof.** In `DICE`, a malicious validator $v'$ can defraud rewards by submitting a copy of VA results received from another honest validator $v$ if and only if this validator submitted the following two values in the `VA_COMMIT` phase [49]: (1) a fingerprint value equal to both $H(\mathbb{VAR}_v)$ and the consensus value $FP_{agreed}$, and (2) a commitment equal to $H(v'||\mathbb{VAR}_v)$. Generating $H(v'||\mathbb{VAR}_v)$ requires accessing the honest validator's $\mathbb{VAR}_v$. However, honest validators in the `VA_COMMIT` phase submit only the fingerprint (including $H(\mathbb{VAR}_v)$, $\mathbb{VAR}_v^{commit}$) and commitment ($H(v||\mathbb{VAR}_v)$), not the complete raw $\mathbb{VAR}_v$. Therefore, the only way to generate $H(v'||\mathbb{VAR}_v)$ in this phase is to guess from the exposed fingerprints and commitments. According to the property of cryptographic hash function $H(\cdot)$, the probability of guessing the inputs to $H(\cdot)$ from its outputs is at most $\frac{1}{2^{\min(hlen,E)}}$ [3, 74], where $E$ is the min-entropy (in bits) of the unknown part of the input space. With $\mathbb{VAR}_v^{commit}$ submitted in the `VA_COMMIT` phase, $E = |\mathbb{VAR}_v \setminus \mathbb{VAR}_v^{commit}| \cdot log_2 N_{input} \geq log_2 N_{input}$. This indicates that, for any given threshold $\epsilon_p$, as long as the inequality in Assumption 1 is satisfied, the probability that malicious validators succeed in fraud attacks is smaller than $\epsilon_p$.

This theorem indicates that it is computationally intractable for any malicious validator to launch a successful fraud attack in `DICE`.

**Theorem 4.** *Under Assumptions 2–3, given any small threshold* $\epsilon_p$, *there exist parameters* $N_t \in \mathbb{Z}^+$, $N_c = \lceil \frac{|\mathbb{V}_{all}|}{2} + \log_{[1-\frac{N_t}{(2|\mathbb{W}^r|-1)N_{lus}}]} \epsilon_p \rceil$ *such that, in the* `VA_COMMIT` *status, the probability that at least one of the honest validators submits correct results for any conflict-unrelated VA sub-task is greater than* $1 - \epsilon_p$.

*Proof.* The complement of the event in Theorem 4 is that none of honest validators proposes correct results for a given conflict-unrelated sub-task in the `VA_COMMIT` phase. Below, we show that with $N_t \in \mathbb{Z}^+$ and $N_c = \lceil \frac{|\mathbb{V}_{all}|}{2} + \log_{[1-\frac{N_t}{(2|\mathbb{W}^r|-1)N_{lus}}]} \epsilon_p \rceil$, the probability that Theorem 4's complementary event occurs is smaller than $\epsilon_p$, indicating the correctness of this theorem.

In `DICE`, the `VA_COMMIT` phase will not end before $N_c$ validators have submitted results for at least $\frac{N_{VA}-N_{CF}}{N_{VA}} N_t$ conflict-unrelated VA sub-tasks. Each honest validator randomly selects the conflict-unrelated sub-tasks for which it submits results, from the VA task forest composed of $N_{VA}$ sub-tasks where $N_{VA} \leq (2|\mathbb{W}^r| - 1)N_{lus}$. Therefore, during the execution time of `VA_COMMIT`, the probability that all honest validators do not submit results for a conflict-unrelated VA sub-task is given by Eq. 13, where $\mathbb{V}_h$ is the set of honest validators who have submitted results for at least $\frac{N_{VA}-N_{CF}}{N_{VA}} N_t$ conflict-unrelated sub-tasks.

$$Pr \leq (1 - \frac{N_t}{N_{VA}})^{|\mathbb{V}_h|} \leq (1 - \frac{N_t}{(2|\mathbb{W}^r| - 1)N_{lus}})^{|\mathbb{V}_h|} \tag{13}$$

Let $N_{malicious}$ denote the total number of malicious validators. It is easy to derive that $N_{malicious} \leq \frac{(1-h)|\mathbb{V}_{all}|E_{avg}}{E_{min}}$. Based on $h \geq \frac{2}{3}$, $\frac{E_{avg}}{E_{min}} \leq \frac{3}{2}$, we have $N_{malicious} < \frac{|\mathbb{V}_{all}|}{2}$. Then, we have Eq. 14–15.

$$|\mathbb{V}_h| \geq N_c - N_{malicious} > N_c - \frac{|\mathbb{V}_{all}|}{2} \tag{14}$$

$$Pr < (1 - \frac{N_t}{(2|\mathbb{W}^r| - 1)N_{lus}})^{N_c - \frac{|\mathbb{V}_{all}|}{2}} \tag{15}$$

This indicates that, if $N_c = \lceil \frac{|\mathbb{V}_{all}|}{2} + \log_{[1-\frac{N_t}{(2|\mathbb{W}^r|-1)N_{lus}}]} \epsilon_p \rceil$, $Pr < \epsilon_p$.

This theorem states that all conflict-unrelated VA sub-tasks will receive results from honest validators with high probability. Thus, if any malicious validator submits poisoned results for those VA sub-tasks, those sub-tasks must become controversial.

**Theorem 5.** *Under Assumptions 2, given any small threshold* $\epsilon_p$, *there exist parameters* $E_c = 18Z_{1-\epsilon_p}^2$, $\epsilon_c = \frac{1+Z_{1-\epsilon_p}\sqrt{\frac{1-h}{E_c}}+Z_{\epsilon_p}\sqrt{\frac{h}{E_c}}}{2}$ *such that the probability that the consensus results for any root or controversial sub-task are produced by honest validators, is larger than* $1 - \epsilon_p$, *where* $Z_{1-\epsilon_p}$, $Z_{\epsilon_p}$ *are the critical values of the standard normal distribution with a significance level of* $1 - \epsilon_p$, $\epsilon_p$, *respectively.*

**Proof.** For any controversial or non-controversial root sub-task, `DICE` adopts committee voting [49, 110] to determine the consensus. Only results that receive more than $E_c \cdot \epsilon_c$ votes will be accepted as the consensus results. As proved in our previous paper [49], for any small threshold $\epsilon_p$, by setting $E_c = 18Z_{1-\epsilon_p}^2$ and $\epsilon_c = \frac{1+Z_{1-\epsilon_p}\cdot\sqrt{\frac{(1-h)}{E_c}}+Z_{\epsilon_p}\cdot\sqrt{\frac{h}{E_c}}}{2}$, the probability that the consensus results are produced by honest validators is larger than $1 - \epsilon_p$, under Assumption 2. This indicates the correctness of Theorem 5.

This theorem states that all root and controversial sub-tasks will finally be resolved with the consensus produced by honest validators with high probability. Theorems 4–5 collectively prove the probability that any malicious validator succeeds in poisoning attacks in `DICE` is lower than $\epsilon_p$.

Theorems 3–5 jointly prove that, with overwhelming probability, TCC can prevent malicious validators from committing fraud and poisoning, thus ensuring that only honest validators produce consensus VA results and `DICE`'s complete FMC process is secure.

**Discussion.** The correctness of Theorems 3–5 is based on Assumptions 1–3. We now discuss how these assumptions are satisfied.

*Designs supporting Assumption 1.* To satisfy the inequality in Assumption 1, i.e., $\min(hlen,\ \log_2 N_{input}) > \log_{\frac{1}{2}} \epsilon_p$, `DICE` incorporates the following designs. Firstly, it sets the parameter $\epsilon_p$ as a

system parameter that cannot be changed after the system is initialized, and typically sets $\epsilon_p = 10^{-12}$ to ensure a negligible success probability for poisoning and fraud attacks. Then, we select SHA-256 as DICE's cryptographic hash function $H(\cdot)$, which provides an output length of 256 bits, and use uint256 as the type of data in $\mathbb{VAR}_v \setminus \mathbb{VAR}_v^{commit}$, which are the unknown inputs to $H(\cdot)$ in the VA_COMMIT status. This setting satisfies that $hlen$ and $\log_2 N_{input}$, both equal to 256, are greater than $\log_{\frac{1}{2}} \epsilon_p$, which approximates to 40 for $\epsilon_p = 10^{-12}$.

*Designs supporting Assumptions 2–3.* To ensure that three critical security preconditions, including (i) sufficiently large validator pool and total stake, $|\mathbb{V}_{all}| > 2\lceil \log_{[1-\frac{N_t}{(2|\mathbb{W}^r|-1)N_{lus}}]} \epsilon_p \rceil$, $E_{total} >> E_c$, (ii) an honest majority, $h \geq \frac{2}{3}$, and (iii) bounded inequality, $\frac{E_{avg}}{E_{min}} \leq \frac{3}{2}$, are well-established and maintained, DICE adopts a carefully designed staking scheme. It sets $E_{\min} = \max(E_c, Punish)$ and requires each validator to stake between $E_{\min}$ and $\frac{3}{2}E_{\min}$ units of cryptocurrency as a security deposit when joining an FMC task. Given any FMC task, only after the registration of $3\lceil \log_{[1-\frac{N_t}{(2|\mathbb{W}^r|-1)N_{lus}}]} \epsilon_p \rceil$ validators, will DICE start the task. During the task runtime, DICE dynamically removes the validators whose staked deposit falls outside the required range $[E_{\min}, \frac{3}{2}E_{\min}]$. If the number of validators meeting the deposit requirement drops below $3\lceil \log_{[1-\frac{N_t}{(2|\mathbb{W}^r|-1)N_{lus}}]} \epsilon_p \rceil$, DICE then restarts participant registration to replenish the validator pool. These designs collectively guarantee the sufficiently large stake and enforce the bounded inequality that serves two critical purposes: (1) to secure the honest-majority assumption by making it infeasible for any individual to control an excessive stake share; (2) to prevent the scenario where an excessively large deposit inflates the minimum number of validators required for security, as derived from Theorem 4, ensuring practical deployability. As a consequence of a sufficiently-large $E_{total}$ and $\frac{E_{avg}}{E_{min}} \leq \frac{3}{2}$, it becomes intractable for malicious validators to control more than one-third of the total stake, which ensures $h \geq \frac{2}{3}$.

In addition to the above designs, DICE further encourages every validator to behave honestly (thus ensure $h \geq \frac{2}{3}$) by locking the validator's security deposit in smart contracts with no reward for a certain period. Suppose that an FMC task is T and a validator $v$ staked cryptocurrency (or has positive amount of rewards) in round $r$ of T. DICE will lock those cryptocurrencies from round $r$ of task T. Validator $v$ is allowed to withdraw those cryptocurrencies only if it completes at least $N_t \cdot \epsilon_{lock}$ new VA sub-tasks in task T (or its subsequent FMC tasks) and is not found to behave maliciously. Under this design, for any individual with a large stake in DICE, it ought to stay honest when choosing between (i) poisoning the aggregated results and deceiving the model requester and (ii) staying honest (producing the correct aggregated results and being rewarded for doing so). If it chooses to poison the VA results and deceive the model requester, the model requester will not submit new FL tasks anymore, leaving its staked security deposit either locked in the system with no reward or burnt when its malicious behavior is detected. Therefore, staying honest favors the validator who has staked a large amount of security deposit to gain more profit than any other validator in the system.

| $\lvert\mathbb{W}^r\rvert$ | 10 | 30 | 50 | 70 | 100 | 10 | 30 | 50 | 70 | 100 |
|---|---|---|---|---|---|---|---|---|---|---|
| $N_t$ | $O(1)$ | | | | | $O(\log\log\lvert\mathbb{W}^r\rvert)$ | | | | |
| $N_c$ | 1278 | 4041 | 6805 | 9568 | 13712 | 1278 | 4041 | 6805 | 9568 | 6839 |
| $N_t$ | $O(\log\lvert\mathbb{W}^r\rvert)$ | | | | | $O(\sqrt{\lvert\mathbb{W}^r\rvert})$ | | | | |
| $N_c$ | 622 | 1324 | 1675 | 2366 | 2715 | 402 | 781 | 942 | 1166 | 1340 |

**Table 3: Variation of $N_c$ with $N_t$, given $\epsilon_p = 10^{-12}$, $N_{lus} = 1$.**

## 4.3 Trade-off between DICE's Efficiency, Scalability, and Required Validator Quantity

We finally discuss a fundamental trade-off revealed from preceding analyses. At the end of §4.1, we show that DICE can reduce per-validator computational complexity from linear to sub-linear, and even to constant when $N_t$ is set to $O(1)$. This suggests a straightforward path to optimize scalability: minimize $N_t$ to enable even the most resource-constrained devices to participate. However, Theorem 4 reveals a hidden cost of this approach: as $N_t$ decreases, the minimum number of validators required for security ($N_c$) increases.

This inverse relationship creates a fundamental trade-off between two desirable but conflicting goals: minimizing per-validator workload (to attract maximum participation) and minimizing the required validator pool size (to accommodate ecosystems with limited participants). Prior BCFL paradigms [42, 47, 56, 56, 63, 64, 69, 115] did not encounter this trade-off because they never reduced per-validator complexity. They guarantee security with a fixed validator set, but at the cost of linear complexity that excludes resource-constrained devices, leaving a critical gap: how to scale FMC to hundreds of workers or more without assuming all validators possess enterprise-grade hardware.

DICE navigates this trade-off by using $N_t$ as a tuning knob between two regimes: minimizing individual workload (favoring small $N_t$, large $N_c$) and reducing validator pool requirements (favoring larger $N_t$, smaller $N_c$). Table 3 quantifies the relationship between $N_c$ and $N_t$ as the number of workers increases. When the ecosystem can recruit enough validators to meet the threshold $N_c$, a small $N_t$ (e.g., constant) is recommended to minimize per-validator complexity and maximize efficiency. Conversely, if validator recruitment is constrained, a larger $N_t$ (e.g., $\lceil \log|\mathbb{W}^r| \rceil$ or $\lceil \sqrt{|\mathbb{W}^r|} \rceil$) is preferred. For most deployment scenarios, we recommend $N_t = \lceil \log|\mathbb{W}^r| \rceil$ to achieve: (i) *sub-linear per-validator complexity*, substantially lower than the linear complexity of prior paradigms; (ii) *manageable validator pool growth*, where $N_c$ increases slowly with the number of workers; and (iii) *provable security*.

In summary, DICE maintains provable security while achieving the optimal sub-linear per-validator computational complexity, which shows superior efficiency and scalability over prior BCFL paradigms. The tunable parameter $N_t$ offers flexibility to adapt this advantage to practical factors like the actual participant scale, security constraints, and individual resource constraints.

# 5 EXPERIMENTS

This section presents a comprehensive evaluation of DICE, structured into three parts to systematically assess its performance under varying conditions. §5.2 evaluates DICE with *honest but hardware-constrained* participants, against an ideal but infeasible scenario

| Settings | Vision FMC Tasks | Language FMC Tasks |
|---|---|---|
| Type | Image Classification | NER or RE |
| Dataset | FedISIC (FP), CIFAR100, DermaMNIST | 2018n2c2 (FP), BC5-disease, JNLPBA, DDI, GAD, ChemProt |
| $\lvert\mathbb{W}_{all}\rvert, \lvert\mathbb{V}_{all}\rvert$ | 100, 100 | 50, 100 |

**Table 4: Experimental task settings. FP means that the task contains a beforehand further pretraining step.**

where all participants are honest and hardware-unlimited, establishing DICE's performance baseline. §5.3 compares DICE against the current BCFL paradigm in realistic scenarios involving *both hardware-constrained and malicious* participants, demonstrating DICE's holistic advantage. Finally, §5.4 studies DICE's flexibility in integrating with parameter-efficient fine-tuning (PEFT) techniques and tuning $N_t$ to further lower the entry barrier of FMC in Web3.

## 5.1 Experimental Settings

Following our previous paper [49], all experiments are conducted on a private Ethereum blockchain network, called private Ethereum testnet, that is deployed on our in-house private cloud to avoid the real economic costs introduced by invoking smart contracts and making transactions on the public Ethereum network, i.e., Ethereum mainnet. We refer readers to our previous paper [49] for more details about blockchain configuration. Below, we detail the configuration of FMC tasks, participants, the BCFL baselines used for comparison, and evaluation metrics.

*Task Settings.* Our experiments are conducted on nine FMC tasks summarized in Table 4. The vision tasks customize ViT [97] for image classification on three widely-used datasets in FL research [2, 6, 52, 60, 68, 89, 107, 108]. The language tasks select BERT [20, 94] as the FM to be customized for either named entity recognition or relation extraction on six biomedical domain datasets used in previous work [28, 41, 66, 67]. For both vision and language tasks, we select one biomedical dataset to evaluate the performance of fine-tuning FM on that dataset with a beforehand further-pretraining step. We use MedMNIST [102, 103], a large-scale collection of standardized biomedical images, and PubMed [28], a large-scale biomedical text dataset, for further-pretraining in vision and language tasks, respectively.

*Participant Configuration.* We deploy 100 workers and 100 validators for vision FMC tasks, and 50 workers and 100 validators for language FMC tasks, along with one model requester who submits all FMC tasks. Each participant runs in a dedicated virtual machine hosted in our in-house private cloud. We associate each participant with a unique Ethereum testnet account. Each worker is allocated a subset of training samples from the FMC task dataset. To simulate the non-IID data distribution prevalent in real-world Web3 applications, we allocate sample sizes across workers following a Dirichlet distribution and diversify label distributions based on the number of unique labels in each dataset. Validators have access to a held-out validation dataset to evaluate worker updates and assess contributions. For datasets that include dedicated validation samples, we use them directly. In cases where such samples are unavailable, we split the test set in half, using one half for validation and the other for final model evaluation.

Unless otherwise stated, participants in each experiment are heterogeneous in hardware resources and may exhibit malicious behavior. The hardware constraints for workers and validators follow the configurations introduced in §5.2.1 and §5.2.2, respectively. Regarding adversarial settings, we designate 20% of workers and 20% of validators as malicious, with the remaining participants being honest. The only exception is experiments in §5.2, where we configure all participants as honest and constrain only one participant role to establish DICE's performance baseline.

Each malicious worker randomly chooses between two attack types in each round: (i) fraud attack, where it copies an honest worker's update to illegitimately claim rewards; or (ii) poisoning attack, where it submits updates trained on mislabeled backdoor examples [5, 9]. A malicious worker cannot launch both attacks in a single round, as DICE permits only one update per worker per round.

Each malicious validator submits poisoned VA results, generated by assigning random scores above the discard threshold to each malicious worker's update. We focus exclusively on poisoning attacks from malicious validators, as launching fraud attacks is computationally intractable for them according to Theorem 3. All validators stake the same amount of security deposit in our experiments, making the ratio of malicious validators' stake to the total stake equal to their proportion.

*BCFL baseline Configuration.* We implement the current BCFL paradigm based on Refiner [49], a state-of-the-art BCFL algorithm, with two key justifications. First, Refiner is designed under the same threat model as DICE, addressing both poisoning and fraud attacks. In contrast, most BCFL algorithms [15, 19, 22, 38, 46, 59, 72], including BlockFL [19], FedCoin [51], BAFFLE [71], BlockFLA[19], DeepChain [95], BFLC [47], primarily address only poisoning attacks, making them unsuitable for a direct comparison under our comprehensive threat model. Second, Refiner shares the resource-intensive operational assumption common to all existing BCFL algorithms [19, 22, 46, 72, 95], requiring each worker to train a full model and each validator to fully process every worker's update. Therefore, implementing BCFL based on Refiner allows us to faithfully represent the current BCFL paradigm while ensuring a fair and meaningful comparison with DICE.

*Evaluation Metrics.* The evaluation metrics used in our experiments include (1) Efficiency Metrics: $S_{LM}$ (MB), the minimum size of local models deployed on the workers, $|\theta_r^w|$ (MB), the maximum size of tunable parameters on the minimum local model, $S_{VM}$ (MB), the minimum size of validation models deployed on the validators, $N_{tmin}$, the minimum number of VA sub-tasks that a validator should accomplish to achieve global consensus on the entire VA task; (2) Security Metrics: $RMW$ and $RMV$, the proportion of total rewards successfully claimed by malicious workers and validators, respectively; (3) Effectiveness Metrics: $ACC$ and $F1$, the accuracy and F1 score of the customized global FM on downstream tasks. We run each experiment three times repeatedly with different random seeds and report the average of the results of the three runs.

Note that our efficiency metrics are selected based on empirical results from previous work [25, 31, 40, 62, 92]. Under controlled settings such as fixed optimizer, batch size, sequence length, and hardware, etc., $S_{LM}$ determines the minimum computation and activation memory costs of forward-pass on a worker's models,

| Methods | $\mathbb{W}_{all}$ | $S_{LM} \downarrow$ | $\lvert\theta_r^w\rvert \downarrow$ | ACC ↑ | | | $S_{LM} \downarrow$ | $\lvert\theta_r^w\rvert \downarrow$ | F1 ↑ | | | $S_{LM} \downarrow$ | $\lvert\theta_r^w\rvert \downarrow$ | F1 ↑ | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | Vision Tasks | | FedISIC | CIFAR100 | DermaMNIST | NER Tasks | | n2c2 | JNLPBA | BC5-d | RE Tasks | | DDI | GAD | ChemProt |
| BCFL | ✗ | 100.0% | 100.0% | 79.5% | 94.0% | 82.9% | 100.0% | 100.0% | 90.7% | 98.5% | 89.5% | 100.0% | 100.0% | 44.3% | 92.7% | 13.8% |
| +Lora | | 101.6% | 1.7% | 75.7% | 91.5% | 78.3% | 101.2% | 1.6% | 86.4% | 93.5% | 85.2% | 101.2% | 2.2% | 43.8% | 87.4% | 13.1% |
| +BitFit | | 100.0% | **0.2%** | 71.5% | 94.8% | 75.2% | 100.0% | **0.1%** | 81.6% | 59.3% | 84.5% | 100.0% | **0.8%** | 42.2% | 76.3% | 23.0% |
| +Prune | ✓ | 50.4% | 50.0% | 95.9% | 90.9% | 99.6% | **60.9%** | 50.0% | 97.4% | 98.2% | 98.5% | **61.1%** | 50.4% | 93.3% | 99.6% | 78.8% |
| +KD | | **3.3%** | 3.1% | 91.7% | 80.3% | 91.2% | | | 97.1% | 99.0% | **99.7%** | | | 92.8% | **99.9%** | 97.6% |
| +T-Only | ✓ | 50.4% | 41.6% | 73.8% | 89.6% | 92.6% | **60.9%** | 41.7% | 97.2% | 96.3% | 95.3% | **61.1%** | 42.1% | 81.6% | 91.2% | 79.0% |
| +B-Only | | | | 95.9% | 86.8% | 96.2% | | | 96.1% | 96.1% | 98.3% | | | 86.9% | 97.2% | 80.8% |
| +OT | | | | 87.1% | 96.0% | 92.0% | | | 96.9% | 96.3% | 97.8% | | | 76.2% | 91.3% | 79.0% |
| +DyTB | | | | 91.7% | 90.2% | 92.6% | | | 97.2% | 97.6% | 94.6% | | | 93.1% | 79.7% | 79.0% |
| +DyBT | | | | 96.0% | 82.0% | 96.2% | | | 96.1% | 96.1% | 98.4% | | | 86.9% | 94.7% | 80.8% |
| +DyOT | | | | 91.3% | 96.2% | 95.1% | | | 96.9% | 96.3% | 96.7% | | | 88.5% | 95.3% | 79.0% |
| DICE | ✓ | 50.4% | 41.6% | **98.5%** | **98.5%** | **99.8%** | **60.9%** | 41.7% | **98.5%** | **99.2%** | 98.8% | **61.1%** | 42.1% | **96.6%** | 99.7% | **99.5%** |

**Table 5: Results of handling hardware-constrained workers on vision and language (NER, RE) FMC tasks. Values are presented as percentages relative to the ideal baseline (= 100%). Top-1 and top-2 results are marked in bold and underlined, respectively.**

while $|\theta_r^w|$ dictates that model's gradient computation and memory costs as well as communication costs. $S_{VM}$ is the primary determinant of the minimum computational and memory costs for each VA sub-task. The product $S_{VM} \times N_{tmin}$ thus determines the lower bound of the total cost to produce VA consensus results. The metrics $S_{LM} + |\theta_r^w|$ and $S_{VM} \times N_{tmin}$ serve as hardware- and dataset-agnostic proxies of the minimum computational, memory, and communication costs for workers and validators to engage in FMC in Web3.

## 5.2 Efficiency Performance

We first evaluate the efficiency of DICE with honest but hardware-constrained participants against an ideal scenario where each worker and validator have sufficient resources to train a full FM and execute the entire VA task, respectively. To isolate the impact of each participant role's constraints, we conduct two controlled experiments: one with only workers constrained (§5.2.1) and another with only validators constrained (§5.2.2).

*5.2.1 Performance with Hardware-constrained Workers.* This experiment evaluates DICE's performance when workers are hardware-constrained, while validators have abundant resources. We model the constraint by setting the number of deployable T-layers on a worker as $[\frac{L_G}{2} \cdot (1 + \frac{|\mathbb{D}_w| - D_{min}}{D_{max} - D_{min}})]$, where $D_{max}$ and $D_{min}$ are the maximum and minimum sizes of a worker's training dataset, respectively. This simulates a scenario where workers with abundant task-specific data cannot deploy a full FM due to hardware limitations. For comparison, we report the results of eleven baselines, including the current *BCFL* paradigm [49, 95, 110] which excludes hardware-constrained workers due to full FM forward–backward requirement, and its enhanced version powered by representative PEFT techniques (*LoRA*[32] and *BitFit* [8]), model compression techniques (*Prune* [33, 53, 99] and *KD* [30, 37, 77, 98]), static partial FM training techniques (*T-Only*, *B-Only*, *OT*) [75, 99], and unidirectional dynamic training strategies (*DyTB*, *DyBT*, *DyOT*) [75, 99].

Table 5 presents the results of this experiment. Across all FMC tasks, DICE consistently achieves a substantial reduction in computational and memory costs, reducing the worker-side model size ($S_{LM}$) by 40–50% and the size of tunable parameters ($|\theta_r^w|$) by nearly 60%, while maintaining 97% or more of the ideal performance. In contrast, the current BCFL paradigm and its PEFT-enhanced (+Lora/Bitfit) variant sacrifice over 10% of ideal model performance in most tasks, performing the worst among all baselines, since they cannot engage resource-constrained workers who cannot deploy a full FM to contribute their task-specific data. Compression-enhanced variants (+Prune/KD), though effective at solving this issue, suffer significant degradation (5%–20% ACC/F1 drop) on some challenging tasks, such as FedISIC, CIFAR100, and DDI, as compression inherently limits the original FM's representational capacity for tasks with complex data distributions. The remaining six baselines exhibit unstable FMC performance across tasks due to either sub-optimal fixed patterns from static training strategies (T-Only, B-Only, OT) or heuristic, non-adaptive limitations from unidirectional dynamic strategies (DyTB, DyBT, DyOT). DICE addresses the limitations of the nine baselines by its DPL scheme, which (i) engages hardware-limited workers in FMC, (ii) preserves the complete FM architecture for uncompromised inference, and (iii) dynamically adjusts the parameters to be trained in a data- and hardware-aware manner. Thus, DICE not only closely approaches ideal performance under stringent hardware constraints but also delivers the most reliable and resource-efficient training process across diverse tasks.

*5.2.2 Performance with Hardware-constrained Validators.* This experiment evaluates DICE's performance when validators are hardware-constrained, while workers have abundant resources to train a full FM. We set the number of T-layers deployable on each validator to follow a normal distribution $\mathcal{N}(\frac{3L_G}{4}, \frac{L_G^2}{144})$. This simulates a common case where validators have heterogeneous hardware conditions and might not fully deploy the worker's model update due to hardware constraints ($L_{vm} < L_{wm} = L_G$). For comparison, we report the same set of baselines as in the previous subsection, including the prior BCFL paradigm and its PEFT-enhanced or compression-enhanced variants. We exclude static partial FM training techniques and unidirectional dynamic training strategies, as they are specifically designed for worker-constrained scenarios, whereas this experiment focuses on validator-side constraints.

Table 6 presents the results of experiments under hardware-constrained validators. Compared to the current BCFL paradigm,

| Methods | $\mathbb{V}_{all}$ | $S_{VM}\downarrow$ | $N_{tmin}\downarrow$ | ACC ↑ | | | $S_{VM}\downarrow$ | $N_{tmin}\downarrow$ | F1 ↑ | | | $S_{VM}\downarrow$ | $N_{tmin}\downarrow$ | F1 ↑ | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | Vision Tasks | | FedISIC | CIFAR100 | DermaMNIST | NER Tasks | | n2c2 | JNLPBA | BC5-d | RE Tasks | | DDI | GAD | ChemProt |
| BCFL | ✗ | 100.0% | 100.0% | **100.0%** | **100.0%** | **100.0%** | 100.0% | 100.0% | **100.0%** | **100.0%** | **100.0%** | 100.0% | 100.0% | **100.0%** | **100.0%** | **100.0%** |
| +Lora | | | | 97.2% | 98.4% | 97.5% | | | 96.3% | 96.0% | 97.2% | | | 96.9% | 97.3% | 94.9% |
| +BitFit | | | | 89.9% | 92.0% | 90.7% | | | 91.0% | 93.3% | 87.5% | | | 90.1% | 92.3% | 89.7% |
| +Prune | ✓ | 50.4% | | 95.9% | 90.9% | 99.6% | **60.9%** | | 97.4% | 98.2% | 98.5% | **61.1%** | | 93.3% | 99.6% | 78.8% |
| +KD | | **3.3%** | | 91.7% | 80.3% | 91.2% | | | 97.1% | 99.0% | 99.7% | | | 92.8% | 99.9% | 97.6% |
| DICE | ✓ | 50.4% | **2.6%** | 98.1% | 99.4% | 99.8% | **60.9%** | **3.5%** | 98.1% | 99.2% | 98.8% | **61.1%** | **3.5%** | 97.6% | 98.2% | 98.7% |

**Table 6: Results of handling hardware-constrained validators on vision and language FMC tasks.**

DICE reduces the minimum size of the model deployed on any validator ($S_{VM}$) by 40–50% and the minimum number of VA sub-tasks that each validator must complete for consensus ($N_{tmin}$) by an order of magnitude, thus resulting in a substantially smaller total cost bound $S_{VM} \times N_{tmin}$ for validators, while maintaining competitive performance of customized FMs (more than 98% of ideal performance) in all tasks. Notably, validator-side constraints do not directly impact model performance, since validators only verify aggregated updates rather than contribute training data. Therefore, BCFL and its PEFT-enhanced variants show much better ACC/F1 than they achieved in previous worker-constrained experiments. However, these baselines require full forward-backward computation and multi-update verification, which would exclude many honest but resource-constrained Web3 participants from engaging as validators in real-world FMC tasks. Though the compression-enhanced variants overcome this limitation, they may compromise the model utility as explained in previous experiments. In contrast, DICE achieves substantial efficiency gains and trivial loss in ACC/F1 across different tasks through its MSV and TCC schemes. These results validate the effectiveness of these schemes in achieving a practical balance that allows an increasing number of honest but resource-constrained validators to contribute effectively without compromising the final model performance.

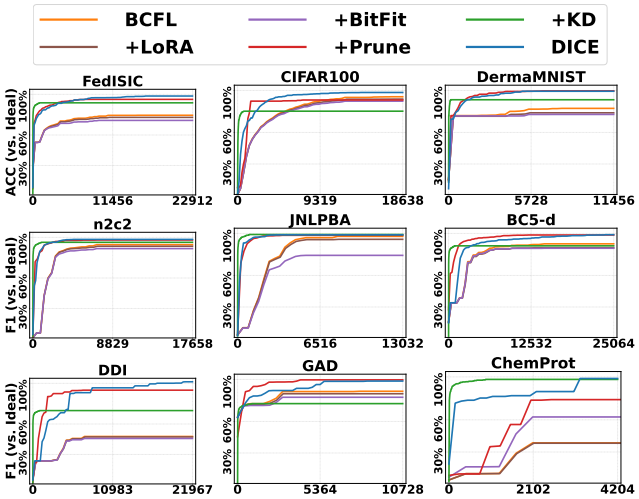



**Figure 6: Performance of FM customized by different baselines. X-axis: total size of models loaded by the most hardware-constrained worker or validator. These participants deploy models of (near-)identical size for each baseline in these experiments, so their x-axes coincide.**

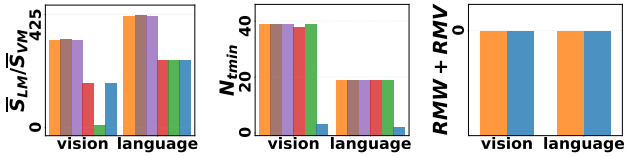



**Figure 7: Per-participant workload ($S_{LM}/S_{VM}$, $N_{v,t}$) and security performance ($RMW + RMV$) under different baselines.**

## 5.3 Comparison with Current BCFL Paradigm

This subsection presents a comprehensive comparison between DICE and the current BCFL paradigm under a realistic scenario that concurrently incorporates hardware-constrained and malicious participants. Following the setups in previous experiments, we report representative baselines including BCFL and its PEFT-enhanced and compression-enhanced variants.

Fig. 6 shows how the performance of FMs customized by different baselines varies as the participant costs accumulate. Consistent with the findings in §5.2.1–5.2.2, DICE achieves a superior performance frontier, attaining near-ideal ACC/F1 at significantly lower cost than BCFL and its PEFT-enhanced variants, while avoiding the apparent performance degradation observed in compression-enhanced variants on complex tasks (e.g., +Prune variant on CIFAR100, DDI, ChemProt; +KD variant on FedISIC, GAD).

Fig. 7 zooms into the results to examine how DICE addresses the limitation of the existing BCFL paradigm and its variants through four synergistic techniques (DPL, MSV, TCC, EDI). The first sub-figure shows that DICE drastically reduces the size of the model that each worker or validator should load for training or VA through DPL and MSV. The second sub-figure shows that DICE significantly reduces the number of VA sub-tasks each honest validator must accomplish for consensus. This is because TCC decomposes the monolithic VA task, enabling validators to execute only a few sub-tasks and invoking committee voting only for root and controversial sub-tasks, unlike the current BCFL paradigm, which mandates every validator to execute and vote on the entire task. Together, the two sub-figures explain DICE's slower accumulated costs on both workers and validators observed in Fig. 6. Note that in realistic deployments, EDI, which works jointly with TCC to contain validator-side cost inflation, would disincentivize malicious validators, reduce the occurrence of conflicts, and thereby amplify the

| Methods | $S_{LM} \downarrow$ | $\lvert\theta_r^w\rvert \downarrow$ | ACC ↑ | | |
|---|---|---|---|---|---|
| | | | FedISIC | CIFAR100 | DermaMNIST |
| DICE | 165.2±0.1 | 135.2±0.1 | 0.8910 | 0.9039 | 0.8575 |
| +LoRA | 167.3±0.1 | 2.1±0.1 | 0.8465 | 0.8620 | 0.8210 |
| +BitFit | 165.2±0.1 | 0.2±0.1 | 0.8025 | 0.8150 | 0.7780 |
| | | | F1 ↑ | | |
| | | | n2c2 | JNLPBA | BC5-d |
| DICE | 253.2±0.1 | 135.2±0.1 | 0.8290 | 0.7880 | 0.9410 |
| +LoRA | 255.3±0.1 | 2.1±0.1 | 0.7980 | 0.7550 | 0.9170 |
| +BitFit | 253.2±0.1 | 0.2±0.1 | 0.7750 | 0.7320 | 0.8980 |
| | | | DDI | GAD | ChemProt |
| DICE | 255.4±0.1 | 137.5±0.1 | 0.8220 | 0.7250 | 0.5680 |
| +LoRA | 257.5±0.1 | 2.1±0.1 | 0.7910 | 0.6960 | 0.5430 |
| +BitFit | 255.4±0.1 | 0.2±0.1 | 0.7680 | 0.6750 | 0.5250 |

**Table 7: Complete results of DICE integrated with PEFT on different FMC tasks.**

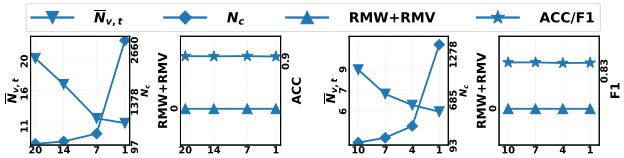
per-validator efficiency gains demonstrated here. Finally, the last sub-figure demonstrates that DICE achieves full reward protection, meaning all attacks, including poisoning or fraud attempts, are successfully thwarted. This explains that DICE's near-ideal model performance observed in Fig. 6 is achieved under the same strong security guarantees as Refiner, confirming that its efficiency gains do not come at the cost of security or model utility.

Collectively, results in Fig. 6–7 not only further corroborate the findings in §5.2.1–5.2.2, but also confirm that DICE establishes a practical paradigm that simultaneously lowers participation barriers, controls operational expenses, and maintains robust security and model utility for FMC in real-world Web3 applications.

## 5.4 Flexibility Performance

Finally, we conduct additional experiments to study the compatibility of DICE with PEFT, which can further reduce workers' computation costs, and the effects of varying $N_t$ to further reduce validators' computation costs and lower the global consensus's latency.

*Integration with PEFT Techniques.* Table 7 shows the results of DICE integrated with PEFT techniques. The results reveal two key patterns. First, integrating PEFT with DICE substantially reduces the update size $|\theta_r^w|$, e.g., from 135.0+ MB to 2.0+ MB with LoRA, thus lowering the worker's *backward* computation and memory costs and system-level communication overhead. However, these efficiency gains come with varying impacts on FMC performance. LoRA maintain competitive ACC/F1, while BitFit incurs a more noticeable drop. Second, all PEFT techniques do not reduce, and in some cases (e.g., with LoRA), even increase the size of worker-side models ($S_{LM}$) due to additional inserted parameters, thus enlarging the worker's *forward* computation and memory costs. This highlights a fundamental limitation of PEFT for engaging hardware-constrained participants in existing BCFL paradigms: it cannot lower $S_{LM}$, the primary barrier when loading the full FM exceeds device memory capacity. DICE overcomes this through DPL, which simultaneously reduces $S_{LM}$ and $|\theta_r^w|$. Its compatibility with PEFT offers an additional orthogonal path to further optimize worker-side costs and system-level communication overhead, providing practitioners with the flexibility to balance efficiency and performance under diverse resource constraints.




(a) FedISIC . (b) n2c2.

**Figure 8: Effects of setting different $N_t$. X-axis: $N_t$.**

*Effects of Different $N_t$.* Fig. 8 demonstrates that, by varying $N_t$, DICE offers tunable efficiency while maintaining robust security and model utility. The first and third sub-figures show that, as $N_t$ decreases, the individual validator's workload $\overline{N}_{v,t}$ drops, while the required validator-pool size $N_c$ rises, validating our theoretical analysis in §4.3. The second and fourth sub-figures reveal that, when the validator pool satisfies the derived $N_c$ threshold, stolen rewards, $RMW + RMV$, remain zero and final model ACC/F1 stays consistently high across all $N_t$ values. We note that the complete results across all other vision and language datasets exhibit identical trends for $\overline{N}_{v,t}$ and $N_c$, as these metrics depend solely on $N_t$, which is varied identically across tasks of the same type, and the number of active workers in each round, which we set constant for tasks of the same type. Additionally, security guarantees ($RMW + MV = 0$) consistently hold across all datasets whenever the validator pool meets the derived $N_c$ threshold, as demonstrated in the representative tasks in Fig. 8. The only task-specific variation lies in the absolute ACC/F1 values, which remain stable across different $N_t$ settings. These results and findings collectively confirm that practitioners can freely choose $N_t$ based on validator availability and resource constraints without compromising security or model utility across diverse tasks.

## 5.5 Customizing Dinov and Qwen

We further evaluate DICE's generalizability by customizing foundation models at a larger scale than BERT and ViT used in the main experiments. Specifically, we replicate the comparative study in § 5.3, the most stringent scenario with both hardware-constrained and malicious participants, using Dinov2 [65] for vision tasks and Qwen2.5 [84] for language tasks.

Fig. 9–10 present the results of this experiment. The results consistently reproduce all key findings from § 5.3. DICE achieves a superior performance frontier compared to existing BCFL paradigms and its enhanced variants, attaining higher accuracy/F1 at significantly lower participant costs across different tasks. The KD baseline is excluded in this extended evaluation, as Dinov2 and Qwen2.5 lack publicly available distilled, a common limitation of KD-based approaches that often require task-specific teacher-student pre-training and are not readily applicable to new architectures. DICE's four synergistic techniques, DPL, MSV, TCC, and EDI, continue to effectively reduce the model size loaded by workers and validators, decrease validators' verification workload, and maintain full reward protection. These consistent results across different model families demonstrate that DICE's efficiency gains and security guarantees

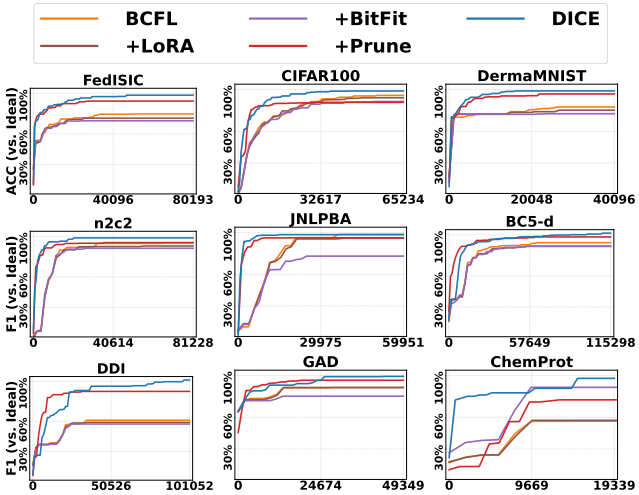



**Figure 9: Performance of Dinov2 and Qwen2.5 customized by different baselines. X-axis: total size of models loaded by the most hardware-constrained worker or validator.**

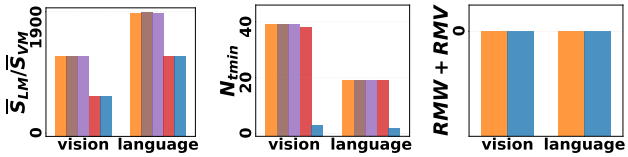



**Figure 10: Per-participant workload ($S_{LM}/S_{VM}$, $N_{v,t}$) and security performance ($RMW + RMV$) under different baselines when customizing Dinov2 and Qwen2.5.**

are architecture-agnostic, reinforcing its potential as a practical framework for decentralized foundation model customization in Web3 environments.

## 6 RELATED WORKS

The work related to DICE mainly falls into three categories: classical federated learning, blockchain-powered federated learning, and efficient FMC techniques.

**Classical federated learning** aims to develop techniques for learning a global model from data distributed among a large number of workers without exposing each worker's data. The first FL algorithm, FedAvg [58], and many of its follow-up works [39, 44, 93] proposed an iterative learning scheme that employs a central server to coordinate workers for learning the global model. Data privacy protection is achieved by transmitting only the model parameters, not data, between the server and the workers. Our work differs from FedAvg and its follow-up works. Those efforts are designed for scenarios where at least one participant, e.g., the server, is trusted. In contrast, our work is designed for Web3 applications where all participants are trustless.

**Blockchain-powered federated learning** aims to involve trustless Web3 participants in FL tasks using blockchain techniques [47, 56, 69]. Considering the self-interested and trustless characteristics of Web3 participants, many studies proposed (1) incentive schemes that reward participants with cryptocurrency in proportion to their contributions, to attract self-interested Web3 participants to engage in FL [49, 113], and (2) well-designed malicious behavior handling schemes which adopt a blockchain network as a reliable data processing system and rely on a consensus algorithm performed by validators in every round of FL to produce correct learning results [49, 113]. The difference between these studies and ours is that, these prior studies are designed for small models with at most tens of millions of parameters, such as CNN [49, 57], LSTM [49, 57], ResNet [11, 57], which can be completely stored and computed on hardware-constrained participants, while DICE is designed for pretrained FMs having at least hundreds of millions of parameters, e.g., BERT [20], ViT [97, 98], Dinov [65], Qwen [84, 100], LLaMa [86, 112], CLIP [70], DICE adopts four novel techniques (DPL, MSV, TCC, EDI) to tackle the efficiency and scalability challenges that prior BCFL algorithms would encounter when applied to FMC tasks.

**Efficient FMC techniques** aim to efficiently further pretrain or fine-tune FMs. These studies typically adopted two types of techniques to achieve high efficiency: (1) parameter-efficient fine-tuning (PEFT) [114], such as LoRA, Adapter, Prompt Tuning, BitFit, (2) model compression, such as off-site tuning [99, 105, 106] and FedBFPT [94]. DICE differs from these studies in two aspects. (1) Most previous work is constrained by a fundamental trade-off between efficiency and model performance. Methods efficient in both computation and memory, e.g., compression-based, typically sacrifice performance, while high-performance methods, e.g., PEFT, only cut computational demands, falling short of mitigating memory overhead. In contrast, DICE aims to achieve maximal FM customization performance by training all parameters, while reducing both computation and memory costs. We note that DICE is orthogonal to PEFT, i.e., can adopt PEFT to further reduce computation costs. (2) Some research is tailored to particular FMC tasks, either further pretraining [94] or fine-tuning [99, 105, 106], without addressing both scenarios. Those studies, for example FedBFPT [94], have strong dependencies on task-specific prior knowledge, e.g., shallower layers play a more important role in pretraining. DICE, conversely, aims to offer a unified non-heuristic solution for diverse FMC tasks and achieves this via a novel DPL scheme.

## 7 CONCLUSION

This paper presented DICE, the first BCFL algorithm specifically designed for efficient and secure FMC in Web3. We systematically identified the efficiency and scalability challenges that prevent existing BCFL paradigms from directly applying to FMC tasks. To overcome these limitations, we introduced and integrated four novel techniques–DPL, MSV, TCC, and EDI–within DICE, and presented a rigorous theoretical analysis of DICE's efficiency and security guarantees. Extensive experiments across both vision and language FMC tasks validated our analysis and demonstrated that DICE significantly reduces both worker- and validator-side costs while maintaining competitive model performance and robust security, consistently outperforming state-of-the-art baselines.

## REFERENCES

[1] 2015. Solidity. https://www.soliditylang.org/
[2] 2022. FedISIC2019. https://owkin.github.io/FLamby/fed_isic.html

[3] Kunbolat Algazy, Kairat Sakan, Nursulu Kapalova, Saule Nyssanbayeva, and Dilmukhanbet Dyusenbayev. 2022. Differential Analysis of a Cryptographic Hashing Algorithm HBC-256. *Applied Sciences* 12, 19 (2022). https://www.mdpi.com/2076-3417/12/19/10173

[4] Pierre-Louis Aublin, Sonia Ben Mokhtar, and Vivien Quéma. 2013. Rbft: Redundant byzantine fault tolerance. In *2013 IEEE 33rd international conference on distributed computing systems*. IEEE, 297–306.

[5] Eugene Bagdasaryan, Andreas Veit, Yiqing Hua, Deborah Estrin, and Vitaly Shmatikov. 2020. How To Backdoor Federated Learning. In *AISTATS*, Vol. 108. 2938–2948.

[6] Yu bao Yan, Chun-Mei Feng, Yuexiang Li, Jinheng Xie, Jun Chen, Mohamed Elhoseiny, Ming Hu, Kaishun Wu, and Lei Zhu. 2025. Temporal Model-Based Federated Active Medical Image Classification. In *International Conference on Medical Image Computing and Computer-Assisted Intervention*. https://api.semanticscholar.org/CorpusID:281830599

[7] Marc Bara. 2025. PromptChain: A Decentralized Web3 Architecture for Managing AI Prompts as Digital Assets. *arXiv preprint arXiv:2507.09579* (2025).

[8] Elad Ben Zaken, Yoav Goldberg, and Shauli Ravfogel. 2022. BitFit: Simple Parameter-efficient Fine-tuning for Transformer-based Masked Language-models. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*. 1–9.

[9] Arjun Nitin Bhagoji, Supriyo Chakraborty, Prateek Mittal, and Seraphin Calo. 2019. Analyzing Federated Learning through an Adversarial Lens. In *ICML*, Vol. 97. 634–643.

[10] Avik Bhattacharjee. 2025. Overview of Model Customization. In *A Practical Guide to Generative AI Using Amazon Bedrock: Building, Deploying, and Securing Generative AI Applications*. Springer, 249–281.

[11] Genqing Bian, Wenjing Qu, and Bilin Shao. 2023. Blockchain-based trusted federated learning with pre-trained models for COVID-19 detection. *Electronics* 12, 9 (2023), 2068.

[12] Rishi Bommasani, Drew A. Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney von Arx, Michael S. Bernstein, Jeannette Bohg, Antoine Bosselut, Emma Brunskill, Erik Brynjolfsson, S. Buch, Dallas Card, Rodrigo Castellon, Niladri S. Chatterji, Annie S. Chen, Kathleen A. Creel, Jared Davis, Dora Demszky, Chris Donahue, Moussa Koulako Bala Doumbouya, Esin Durmus, Stefano Ermon, John Etchemendy, Kawin Ethayarajh, Li Fei-Fei, Chelsea Finn, Trevor Gale, Lauren Gillespie, Karan Goel, Noah D. Goodman, Shelby Grossman, Neel Guha, Tatsunori Hashimoto, Peter Henderson, John Hewitt, Daniel E. Ho, Jenny Hong, Kyle Hsu, Jing Huang, Thomas F. Icard, Saahil Jain, Dan Jurafsky, Pratyusha Kalluri, Siddharth Karamcheti, Geoff Keeling, Fereshte Khani, O. Khattab, Pang Wei Koh, Mark S. Krass, Ranjay Krishna, Rohith Kuditipudi, Ananya Kumar, Faisal Ladhak, Mina Lee, Tony Lee, Jure Leskovec, Isabelle Levent, Xiang Lisa Li, Xuechen Li, Tengyu Ma, Ali Malik, Christopher D. Manning, Suvir Mirchandani, Eric Mitchell, Zanele Munyikwa, Suraj Nair, Avanika Narayan, Deepak Narayanan, Benjamin Newman, Allen Nie, Juan Carlos Niebles, Hamed Nilforoshan, J. F. Nyarko, Giray Ogut, Laurel J. Orr, Isabel Papadimitriou, Joon Sung Park, Chris Piech, Eva Portelance, Christopher Potts, Aditi Raghunathan, Robert Reich, Hongyu Ren, Frieda Rong, Yusuf H. Roohani, Camilo Ruiz, Jack Ryan, Christopher R'e, Dorsa Sadigh, Shiori Sagawa, Keshav Santhanam, Andy Shih, Krishna Parasuram Srinivasan, Alex Tamkin, Rohan Taori, Armin W. Thomas, Florian Tramèr, Rose E. Wang, William Wang, Bohan Wu, Jiajun Wu, Yuhuai Wu, Sang Michael Xie, Michihiro Yasunaga, Jiaxuan You, Matei A. Zaharia, Michael Zhang, Tianyi Zhang, Xikun Zhang, Yuhui Zhang, Lucia Zheng, Kaitlyn Zhou, and Percy Liang. 2021. On the Opportunities and Risks of Foundation Models. *ArXiv* abs/2108.07258 (2021). https://api.semanticscholar.org/CorpusID:237091588

[13] Vitalik Buterin. 2014. A next-generation smart contract and decentralized application platform.

[14] Sarthak Chakraborty and Sandip Chakraborty. 2022. Proof of federated training: accountable cross-network model training and inference. In *2022 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*. IEEE, 1–9.

[15] Yuxia Chang, Chen Fang, and Wenzhuo Sun. 2021. A Blockchain-Based Federated Learning Method for Smart Healthcare. *Computational Intelligence and Neuroscience* 2021, 1 (2021), 4376418.

[16] Guorong Chen, Chao Li, Wei Wang, Li Duan, Bin Wang, Zhen Han, and Xiangliang Zhang. 2025. FairReward: Towards Fair Reward Distribution Using Equity Theory in Blockchain-Based Federated Learning. *IEEE Transactions on Dependable and Secure Computing* 22, 2 (2025), 1612–1626.

[17] Leiming Chen, Dehai Zhao, Liping Tao, Kai Wang, Sibo Qiao, Xingjie Zeng, and Chee Wei Tan. 2025. A Credible and Fair Federated Learning Framework Based on Blockchain. *IEEE Transactions on Artificial Intelligence* 6, 2 (2025), 301–316.

[18] Jiao Dai, He Huang, Yufei Wang, Li Duan, Fuqiang Hu, Dantong Yan, Zhichen Liu, and Wei Wang. 2025. Byzantine-resistant model verification for decentralized federated learning in intelligent transportation systems. *Applied Soft Computing* 169 (2025), 111852.

[19] Harsh Bimal Desai, Mustafa Safa Ozdayi, and Murat Kantarcioglu. 2021. Blockfla: Accountable federated learning via hybrid blockchain architecture. In *Proceedings of the eleventh ACM conference on data and application security and privacy*. 101–112.

[20] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *CoRR* abs/1810.04805 (2018). arXiv:1810.04805 http://arxiv.org/abs/1810.04805

[21] John R. Douceur. 2002. The Sybil Attack. In *Peer-to-Peer Systems*. Springer Berlin Heidelberg, 251–260.

[22] Lei Feng, Yiqi Zhao, Shaoyong Guo, Xuesong Qiu, Wenjing Li, and Peng Yu. 2022. BAFL: A Blockchain-Based Asynchronous Federated Learning Framework . *IEEE Trans. Comput.* 71, 05 (2022), 1092–1103.

[23] Filecoin. [n.d.]. Filecoin Official Website. https://filecoin.io/

[24] Ocean Protocol Foundation. 2020. Ocean Protocol: A Decentralized Data Exchange Protocol to Unlock Data for AI. https://oceanprotocol.com/tech-whitepaper.pdf

[25] Stefanos Georgiou, Maria Kechagia, and Federica Sarro. 2022. Green AI: Do Deep Learning Frameworks Have Different Costs? *2022 IEEE/ACM 44th International Conference on Software Engineering (ICSE)* (2022), 1082–1094. https://api.semanticscholar.org/CorpusID:247614182

[26] Y. Gilad, R. Hemo, S. Micali, G. Vlachos, and N. Zeldovich. 2017. Algorand: Scaling Byzantine Agreements for Cryptocurrencies. In *SOSP*. 51–68.

[27] The Graph. [n.d.]. The Graph Official Website. https://thegraph.com/

[28] Yu Gu, Robert Tinn, Hao Cheng, Michael Lucas, Naoto Usuyama, Xiaodong Liu, Tristan Naumann, Jianfeng Gao, and Hoifung Poon. 2021. Domain-Specific Language Model Pretraining for Biomedical Natural Language Processing. *ACM Trans. Comput. Healthcare* 3, 1 (2021).

[29] Song Han, Huizi Mao, and William J Dally. 2015. Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. *arXiv preprint arXiv:1510.00149* (2015).

[30] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. 2015. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531* (2015).

[31] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. 2022. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556* (2022).

[32] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. 2022. Lora: Low-rank adaptation of large language models. *ICLR* 1, 2 (2022), 3.

[33] Lanxiang Hu, Tajana Rosing, and Hao Zhang. 2025. TrimLLM: Progressive Layer Dropping for Domain-Specific LLMs. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 667–681.

[34] Ahmed Imteaj, Urmish Thakker, Shiqiang Wang, Jian Li, and M Hadi Amini. 2021. A survey on federated learning for resource-constrained IoT devices. *IEEE Internet of Things Journal* 9, 1 (2021), 1–24.

[35] Md Mainul Islam, Mpyana Mwamba Merlec, and Hoh Peter In. 2022. A comparative analysis of proof-of-authority consensus algorithms: Aura vs Clique. In *2022 IEEE International Conference on Services Computing (SCC)*. IEEE, 327–332.

[36] Xu Ji, Hao-Tian Wu, Ting Cui, Yiqun Zhang, and Lingling Xu. 2024. FedGC: Federated Learning on Non-IID Data via Learning from Good Clients. In *Chinese Conference on Pattern Recognition and Computer Vision (PRCV)*. Springer, 181–194.

[37] Xiaoqi Jiao, Yichun Yin, Lifeng Shang, Xin Jiang, Xiao Chen, Linlin Li, Fang Wang, and Qun Liu. 2020. Tinybert: Distilling bert for natural language understanding. In *Findings of the association for computational linguistics: EMNLP 2020*. 4163–4174.

[38] Aditya Pribadi Kalapaaking, Ibrahim Khalil, and Xun Yi. 2023. Blockchain-based federated learning with SMPC model verification against poisoning attack for healthcare systems. *IEEE Transactions on Emerging Topics in Computing* 12, 1 (2023), 269–280.

[39] Sai Praneeth Karimireddy, S. Kale, M. Mohri, Sashank J. Reddi, Sebastian U. Stich, and A. T. Suresh. 2020. SCAFFOLD: Stochastic Controlled Averaging for Federated Learning. In *ICML*.

[40] Vijay Anand Korthikanti, Jared Casper, Sangkug Lym, Lawrence McAfee, Michael Andersch, Mohammad Shoeybi, and Bryan Catanzaro. 2023. Reducing activation recomputation in large transformer models. *Proceedings of Machine Learning and Systems* 5 (2023), 341–353.

[41] Jinhyuk Lee, Wonjin Yoon, Sungdong Kim, Donghyeon Kim, Sunkyu Kim, Chan Ho So, and Jaewoo Kang. 2020. BioBERT: a pre-trained biomedical language representation model for biomedical text mining. *Bioinformatics* 36, 4 (2020), 1234–1240.

[42] Dun Li, Dezhi Han, Tien-Hsiung Weng, Zibin Zheng, Hongzhi Li, Han Liu, Arcangelo Castiglione, and Kuan-Ching Li. 2022. Blockchain for federated learning toward secure distributed machine learning systems: a systemic survey. *Soft Computing* 26, 9 (2022), 4423–4440.

[43] Kai Li, Zhicai Zhang, and Fang Fu. 2023. An Incentive Mechanism for Consortium Blockchain-based Cross-Silo Federated Learning. In *2023 IEEE 29th International Conference on Parallel and Distributed Systems (ICPADS)*. 451–458.

[44] Tian Li, Anit Kumar Sahu, Manzil Zaheer, Maziar Sanjabi, Ameet Talwalkar, and Virginia Smith. 2020. Federated Optimization in Heterogeneous Networks. In *MLSys*, Vol. 2. 429–450.

[45] Wenyu Li, Chenglin Feng, Lei Zhang, Hao Xu, Bin Cao, and Muhammad Ali Imran. 2020. A scalable multi-layer PBFT consensus for blockchain. *IEEE Transactions on Parallel and Distributed Systems* 32, 5 (2020), 1146–1160.

[46] Yuzheng Li, Chuan Chen, Nan Liu, Huawei Huang, Zibin Zheng, and Qiang Yan. 2020. A blockchain-based decentralized federated learning framework with committee consensus. *IEEE Network* 35, 1 (2020), 234–241.

[47] Yuzheng Li, Chuan Chen, Nan Liu, Huawei Huang, Zibin Zheng, and Qiang Yan. 2021. A Blockchain-Based Decentralized Federated Learning Framework with Committee Consensus. *IEEE Network* 35, 1 (2021), 234–241.

[48] Tailin Liang, John Glossner, Lei Wang, Shaobo Shi, and Xiaotong Zhang. 2021. Pruning and quantization for deep neural network acceleration: A survey. *Neurocomputing* 461 (2021), 370–403.

[49] Hong Lin, Ke Chen, Dawei Jiang, Lidan Shou, and Gang Chen. 2024. Refiner: a reliable and efficient incentive-driven federated learning system powered by blockchain. *The VLDB Journal* 33, 3 (2024), 807–831.

[50] Hong Lin, Shixin Wan, Zhongle Xie, Ke Chen, Meihui Zhang, Lidan Shou, and Gang Chen. 2025. A Comprehensive Study of Shapley Value in Data Analytics. *Proc. VLDB Endow.* 18, 9 (2025), 3077–3092.

[51] Yuan Liu, Zhengpeng Ai, Shuai Sun, Shuangfeng Zhang, Zelei Liu, and Han Yu. 2020. *FedCoin: A Peer-to-Peer Payment System for Federated Learning*. 125–138.

[52] Charles Lu and Jayasheree Kalpathy-Cramer. 2022. Distribution-Free Federated Learning with Conformal Predictions. arXiv:2110.07661

[53] Yao Lu, Hao Cheng, Yujie Fang, Zeyu Wang, Jiaheng Wei, Dongwei Xu, Qi Xuan, Xiaoniu Yang, and Zhaowei Zhu. 2024. Reassessing layer pruning in llms: New insights and methods. *arXiv preprint arXiv:2411.15558* (2024).

[54] Yuan Lu, Qiang Tang, and Guiling Wang. 2018. On enabling machine learning tasks atop public blockchains: A crowdsourcing approach. In *2018 IEEE international conference on data mining workshops (ICDMW)*. IEEE, 81–88.

[55] Elizabeth Lui and Jiahao Sun. 2025. Bittensor Protocol: The Bitcoin in Decentralized Artificial Intelligence? A Critical and Empirical Analysis. *arXiv preprint arXiv:2507.02951* (2025).

[56] Chuan Ma, Jun Li, Long Shi, Ming Ding, Taotao Wang, Zhu Han, and H. Vincent Poor. 2022. When Federated Learning Meets Blockchain: A New Distributed Learning Paradigm. *IEEE Computational Intelligence Magazine* 17, 3 (2022), 26–33.

[57] T Manoj, Krishnamoorthi Makkithaya, and VG Narendra. 2025. A blockchain-assisted trusted federated learning for smart agriculture. *SN Computer Science* 6, 3 (2025), 221.

[58] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas. 2017. Communication-Efficient Learning of Deep Networks from Decentralized Data. In *AISTATS*, Vol. 54. 1273–1282.

[59] Brandon Alejandro Mosqueda González, Omar Hasan, and Lionel Brunie. 2025. Mitigation of Sybil-based Poisoning Attacks in Permissionless Decentralized Learning. In *2025 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*. IEEE, IEEE, 1–5. https://hal.science/hal-05002718

[60] Noorain Mukhtiar, Adnan Mahmood, Yipeng Zhou, Jian Yang, Jing Teng, and Quan Z. Sheng. 2024. Federated Learning at the Forefront of Fairness: A Multifaceted Perspective. In *International Joint Conference on Artificial Intelligence*. https://api.semanticscholar.org/CorpusID:281484496

[61] Satoshi Nakamoto. 2008. Bitcoin: A Peer-to-Peer Electronic Cash System.

[62] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, et al. 2021. Efficient large-scale language model training on gpu clusters using megatron-lm. In *Proceedings of the international conference for high performance computing, networking, storage and analysis*. 1–15.

[63] Dinh C Nguyen, Ming Ding, Quoc-Viet Pham, Pubudu N Pathirana, Long Bao Le, Aruna Seneviratne, Jun Li, Dusit Niyato, and H Vincent Poor. 2021. Federated learning meets blockchain in edge computing: Opportunities and challenges. *IEEE Internet of Things Journal* 8, 16 (2021), 12806–12825.

[64] Weiguang Ning, Yingjuan Zhu, Caixia Song, Hongxia Li, Lihui Zhu, Jinbao Xie, Tianyu Chen, Tong Xu, Xi Xu, and Jiwei Gao. 2024. Blockchain-Based Federated Learning: A Survey and New Perspectives. *Applied Sciences* 14, 20 (2024).

[65] Maxime Oquab, Timothée Darcet, Theo Moutakanni, Huy V. Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, Russell Howes, Po-Yao Huang, Hu Xu, Vasu Sharma, Shang-Wen Li, Wojciech Galuba, Mike Rabbat, Mido Assran, Nicolas Ballas, Gabriel Synnaeve, Ishan Misra, Herve Jegou, Julien Mairal, Patrick Labatut, Armand Joulin, and Piotr Bojanowski. 2023. DINOv2: Learning Robust Visual Features without Supervision.

[66] Le Peng, Gaoxiang Luo, Sicheng Zhou, Jiandong Chen, Ziyue Xu, Ju Sun, and Rui Zhang. 2024. An in-depth evaluation of federated learning on biomedical natural language processing for information extraction. *npj Digit. Medicine* 7, 1 (2024).

[67] Le Peng, sicheng zhou, jiandong chen, Rui Zhang, Ziyue Xu, and Ju Sun. 2023. A Systematic Evaluation of Federated Learning on Biomedical Natural Language Processing. https://openreview.net/forum?id=pLEQFXACNA

[68] Sara Pieri, Jose Renato Restom, Samuel Horvath, and Hisham Cholakkal. 2023. Handling data heterogeneity via architectural design for federated visual recognition. In *Proceedings of the 37th International Conference on Neural Information Processing Systems*.

[69] Youyang Qu, Md Palash Uddin, Chenquan Gan, Yong Xiang, Longxiang Gao, and John Yearwood. 2022. Blockchain-enabled Federated Learning: A Survey. *ACM Comput. Surv.* 55, 4 (2022), 35.

[70] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. 2021. Learning Transferable Visual Models From Natural Language Supervision. In *International Conference on Machine Learning*. https://api.semanticscholar.org/CorpusID:231591445

[71] Paritosh Ramanan and Kiyoshi Nakayama. 2020. BAFFLE : Blockchain Based Aggregator Free Federated Learning. In *2020 IEEE International Conference on Blockchain (Blockchain)*. 72–81.

[72] Yanli Ren, Mingqi Hu, Zhe Yang, Guorui Feng, and Xinpeng Zhang. 2024. BPFL: Blockchain-based privacy-preserving federated learning against poisoning attack. *Information sciences* 665 (2024), 120377.

[73] Mónica Ribero, Haris Vikalo, and Gustavo De Veciana. 2022. Federated learning under intermittent client availability and time-varying communication constraints. *IEEE Journal of Selected Topics in Signal Processing* 17, 1 (2022), 98–111.

[74] Phillip Rogaway and Thomas Shrimpton. 2004. Cryptographic Hash-Function Basics: Definitions, Implications, and Separations for Preimage Resistance, Second-Preimage Resistance, and Collision Resistance. In *Fast Software Encryption*. Springer Berlin Heidelberg, Berlin, Heidelberg, 371–388.

[75] Hassan Sajjad, Fahim Dalvi, Nadir Durrani, and Preslav Nakov. 2023. On the effect of dropping layers of pre-trained transformer models. *Computer Speech & Language* 77 (2023), 101429.

[76] Fahad Saleh. 2021. Blockchain without waste: Proof-of-stake. *The Review of financial studies* 34, 3 (2021), 1156–1190.

[77] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. 2019. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108* (2019).

[78] Johannes Schneider, Christian Meske, and Pauline Kuss. 2024. Foundation models: A new paradigm for artificial intelligence. *Business & Information Systems Engineering* 66, 2 (2024), 221–231.

[79] Prateek Sethi. 2023. *Reinforcement Learning assisted Adaptive difficulty of Proof of Work (PoW) in Blockchain-enabled Federated Learning*. Ph.D. Dissertation. Virginia Tech.

[80] Meng Shen, Zhehui Tan, Dusit Niyato, Yuzhi Liu, Jiawen Kang, Zehui Xiong, Liehuang Zhu, Wei Wang, and Xuemin Shen. 2024. Artificial intelligence for web 3.0: A comprehensive survey. *Comput. Surveys* 56, 10 (2024), 1–39.

[81] Behnaz Soltani, Venus Haghighi, Adnan Mahmood, Quan Z. Sheng, and Lina Yao. 2022. A survey on participant selection for federated learning in mobile networks. In *Proceedings of the 17th ACM Workshop on Mobility in the Evolving Internet Architecture*. 19–24.

[82] John Sousa, Eduardo Ribeiro, Romulo Bustincio, Lucas Bastos, Renan Morais, Eduardo Cerqueira, and Denis Rosário. 2025. Enhancing robustness in federated learning using minimal repair and dynamic adaptation in a scenario with client failures. *Annals of Telecommunications* (2025), 1–15.

[83] Fabian Stricker, David Bermbach, and Christian Zirpins. 2025. Analyzing the Impact of Participant Failures in Cross-Silo Federated Learning. *arXiv preprint arXiv:2511.14456* (2025).

[84] Qwen Team. 2024. Qwen2.5: A Party of Foundation Models. https://qwenlm.github.io/blog/qwen2.5/

[85] Youliang Tian, Ta Li, Jinbo Xiong, Md Zakirul Alam Bhuiyan, Jianfeng Ma, and Changgen Peng. 2021. A blockchain-based machine learning framework for edge services in IIoT. *IEEE Transactions on Industrial Informatics* 18, 3 (2021), 1918–1929.

[86] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. LLaMA: Open and Efficient Foundation Language Models. *arXiv* (2023). arXiv:2302.13971

[87] Kentaroh Toyoda, Jun Zhao, Allan Neng Sheng Zhang, and P Takis Mathiopoulos. 2020. Blockchain-enabled federated learning with mechanism design. *Ieee Access* 8 (2020), 219744–219756.

[88] Nguyen Truong, Kai Sun, Siyao Wang, Florian Guitton, and YiKe Guo. 2021. Privacy preservation in federated learning: An insightful survey from the GDPR perspective. *Computers & Security* 110 (2021), 102402.

[89] Adam Tupper and Christian Gagn'e. 2025. Personalized Federated Fine-Tuning of Vision Foundation Models for Healthcare. *ArXiv* abs/2510.12741 (2025). https://api.semanticscholar.org/CorpusID:282064942

[90] Irshad Ullah, Xiaoheng Deng, Xinjun Pei, Ping Jiang, and Husnain Mushtaq. 2023. A verifiable and privacy-preserving blockchain-based federated learning approach. *Peer-to-Peer Networking and Applications* 16, 5 (2023), 2256–2270.

[91] Muhammad Habib ur Rehman, Ahmed Mukhtar Dirir, Khaled Salah, Ernesto Damiani, and Davor Svetinovic. 2021. TrustFed: A framework for fair and trustworthy cross-device federated learning in IIoT. *IEEE Transactions on Industrial Informatics* 17, 12 (2021), 8485–8494.

[92] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).

[93] Hongyi Wang, Mikhail Yurochkin, Yuekai Sun, Dimitris Papailiopoulos, and Yasaman Khazaeni. 2020. Federated Learning with Matched Averaging. In *ICLR*.

[94] Xin'ao Wang, Huan Li, Ke Chen, and Lidan Shou. 2023. FedBFPT: An Efficient Federated Learning Framework for Bert Further Pre-training. In *International Joint Conference on Artificial Intelligence*. https://api.semanticscholar.org/CorpusID:260859911

[95] Jiasi Weng, Jian Weng, Jilian Zhang, Ming Li, Yue Zhang, and Weiqi Luo. 2021. DeepChain: Auditable and Privacy-Preserving Deep Learning with Blockchain-Based Incentive. *IEEE Trans. Dependable Secur. Comput.* 18, 5 (2021), 2438–2455.

[96] Gavin Wood. 2014. Ethereum: a secure decentralised generalised transaction ledger. *Ethereum project yellow paper* 151 (2014), 1–32.

[97] Bichen Wu, Chenfeng Xu, Xiaoliang Dai, Alvin Wan, Peizhao Zhang, Zhicheng Yan, Masayoshi Tomizuka, Joseph Gonzalez, Kurt Keutzer, and Peter Vajda. 2020. Visual Transformers: Token-based Image Representation and Processing for Computer Vision. arXiv:2006.03677 [cs.CV]

[98] Kan Wu, Jinnian Zhang, Houwen Peng, Mengchen Liu, Bin Xiao, Jianlong Fu, and Lu Yuan. 2022. Tinyvit: Fast pretraining distillation for small vision transformers. In *European conference on computer vision*. Springer, 68–85.

[99] Guangxuan Xiao, Ji Lin, and Song Han. 2023. Offsite-tuning: Transfer learning without full model. *arXiv preprint arXiv:2302.04870* (2023).

[100] An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, Guanting Dong, Haoran Wei, Huan Lin, Jialong Tang, Jialin Wang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Ma, Jin Xu, Jingren Zhou, Jinze Bai, Jinzheng He, Junyang Lin, Kai Dang, Keming Lu, Keqin Chen, Kexin Yang, Mei Li, Mingfeng Xue, Na Ni, Pei Zhang, Peng Wang, Ru Peng, Rui Men, Ruize Gao, Runji Lin, Shijie Wang, Shuai Bai, Sinan Tan, Tianhang Zhu, Tianhao Li, Tianyu Liu, Wenbin Ge, Xiaodong Deng, Xiaohuan Zhou, Xingzhang Ren, Xinyu Zhang, Xipin Wei, Xuancheng Ren, Yang Fan, Yang Yao, Yichang Zhang, Yu Wan, Yunfei Chu, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zhihao Fan. 2024. Qwen2 Technical Report. *arXiv preprint arXiv:2407.10671* (2024).

[101] Bang Yang, Asif Raza, Yuexian Zou, and Tong Zhang. 2023. Customizing general-purpose foundation models for medical report generation. *arXiv preprint arXiv:2306.05642* (2023).

[102] Jiancheng Yang, Rui Shi, and Bingbing Ni. 2021. MedMNIST Classification Decathlon: A Lightweight AutoML Benchmark for Medical Image Analysis. In *IEEE 18th International Symposium on Biomedical Imaging (ISBI)*. 191–195.

[103] Jiancheng Yang, Rui Shi, Donglai Wei, Zequan Liu, Lin Zhao, Bilian Ke, Hanspeter Pfister, and Bingbing Ni. 2023. MedMNIST v2-A large-scale lightweight benchmark for 2D and 3D biomedical image classification. *Scientific Data* 10, 1 (2023), 41.

[104] Xiaohui Yang and Tianchang Li. 2024. A Blockchain-based federated learning framework for secure aggregation and fair incentives. *Connection Science* 36 (2024). https://api.semanticscholar.org/CorpusID:267970430

[105] Kai Yao, Zhaorui Tan, Penglei Gao, Lichun Li, Kaixin Wu, Yinggui Wang, Yuan Zhao, Yixin Ji, Jianke Zhu, and Wei Wang. 2025. GradOT: Training-free Gradient-preserving Offsite-tuning for Large Language Models. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 5115–5130. https://aclanthology.org/2025.acl-long.255/

[106] Kai Yao, Zhaorui Tan, Tiandi Ye, Lichun Li, Yuan Zhao, Wenyan Liu, Wei Wang, and Jianke Zhu. 2025. ScaleOT: Privacy-utility-scalable Offsite-tuning with Dynamic LayerReplace and Selective Rank Compression. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 22074–22082.

[107] Mang Ye, Wenke Huang, Zekun Shi, He Li, and Bo Du. 2025. Revisiting federated learning with label skew: an over-confidence perspective. *Science China Information Sciences* 68 (2025). https://api.semanticscholar.org/CorpusID:279980636

[108] Musa Yenilmez and Ilhan Aydin. 2025. Particle Swarm Optimized Federated Learning for Efficient Classification of Medical Images. In *2025 15th International Conference on Advanced Computer Information Technologies (ACIT)*. 868–872.

[109] Yufeng Zhan, Jie Zhang, Zicong Hong, Leijie Wu, Peng Li, and Song Guo. 2022. A Survey of Incentive Mechanism Design for Federated Learning. *IEEE Transactions on Emerging Topics in Computing* 10, 2 (2022), 1035–1044.

[110] Bingxue Zhang, Guangguang Lu, Yuncheng Wu, Kunpeng Ren, and Feida Zhu. 2025. Zkfhed: A Verifiable and Scalable Blockchain-Enhanced Federated Learning System. *IEEE Transactions on Knowledge and Data Engineering* 37, 6 (2025), 3841–3854.

[111] Peng Zhang, Gang Liu, Zhiyu Chen, Jianwei Guo, and Peng Liu. 2022. A study of a federated learning framework based on the interstellar file system and blockchain: Private Blockchain Federated Learning. In *2022 3rd International Conference on Computer Vision, Image and Deep Learning & International Conference on Computer Engineering and Applications (CVIDL & ICCEA)*. IEEE, 267–273.

[112] Peiyuan Zhang, Guangtao Zeng, Tianduo Wang, and Wei Lu. 2024. TinyLlama: An Open-Source Small Language Model. arXiv:2401.02385 [cs.CL]

[113] Zhebin Zhang, Dajie Dong, Yuhang Ma, Yilong Ying, Dawei Jiang, Ke Chen, Lidan Shou, and Gang Chen. 2021. Refiner: a reliable incentive-driven federated learning system powered by blockchain. *Proc. VLDB Endow.* 14, 12 (2021), 2659–2662.

[114] Zhuo Zhang, Yuanhang Yang, Yong Dai, Qifan Wang, Yue Yu, Lizhen Qu, and Zenglin Xu. 2023. Fedpetuning: When federated learning meets the parameter-efficient tuning methods of pre-trained language models. In *Annual Meeting of the Association of Computational Linguistics 2023*. Association for Computational Linguistics (ACL), 9963–9977.

[115] Juncen Zhu, Jiannong Cao, Divya Saxena, Shan Jiang, and Houda Ferradi. 2023. Blockchain-empowered federated learning: Challenges, solutions, and future directions. *Comput. Surveys* 55, 11 (2023), 1–31.

[116] ZIRCONTech. 2025. The Convergence of AI and Web3: Building the Future of Decentralized Intelligence. https://zircon.tech/blog/the-convergence-of-ai-and-web3-building-the-future-of-decentralized-intelligence/